



리눅스 마스터 2급

핵심요약

리눅스 마스터 2급

핵심요약

PART 1 리눅스 일반

p.1-4

- 1. 리눅스의 이해
- 2. 리눅스 설치
- 3. 기본 명령어

PART 2 리눅스 운영 및 관리 ①

p.5-9

- 1. 파일 시스템 관련 명령어
- 2. Shell
- 3. 프로세스 관리

PART 3 리눅스 운영 및 관리 ②

p.10-14

- 1. 에디터
- 2. 소프트웨어 설치
- 3. 장치 설정

PART 4 리눅스 활용

p.15-19

- 1. X 윈도우
- 2. 인터넷 활용
- 3. 응용분야

Part 1. 리눅스 일반

1. 리눅스의 이해

1.1 운영 체제(OS)의 개념

운영 체제란 사용자가 컴퓨터를 편리하게 사용할 수 있도록 도와주는 프로그램들의 집합임. 하드디스크에 저장된 전기 신호 형태의 데이터를 문자·아이콘·이미지처럼 사용자가 인식할 수 있는 형태로 변환하는 역할을 함.

컴퓨터의 3대 주요 부품	
부품	특징
CPU	처리·계산 속도가 가장 빠르지만 저장 기능 없음
메모리(RAM)	CPU보다 느리고 하드디스크보다 빠름, 전원이 꺼지면 데이터가 사라짐 (휘발성)
하드디스크 (HDD)	대용량이지만 속도가 느림, 전원이 꺼져도 데이터 보존 (비휘발성)
SSD	HDD를 대체하는 비휘발성 저장 장치로, HDD보다 훨씬 빠르면서도 전원이 꺼져도 데이터를 보존함

CPU·메모리·하드디스크는 메인보드를 매개로 연결되어 데이터를 주고받으며 동작함. 프로그램은 하드디스크에 저장되어 있다가 실행 시 메모리에 올라가고 CPU가 이를 읽어 처리함.

운영 체제의 역할

- 하드웨어 자원(CPU·메모리·디스크 등)을 효율적으로 배분
- 물리적 장치를 사용자가 쉽게 제어할 수 있도록 인터페이스 제공
- 프로그램 실행·관리, 파일 관리, 보안 관리 등을 수행

1.2 리눅스의 특징

1. 오픈 소스 운영 체제

- 소스 코드와 관련 자료가 모두 공개되어 누구나 무료로 열람·수정·재배포할 수 있음
- 사용자가 원하는 기능을 추가해 자신만의 리눅스를 만들어 쓸 수 있음

2. 멀티 유저 / 멀티 태스킹

- 멀티 유저**: 한 대의 컴퓨터를 여러 사용자가 동시에 사용 가능함
- 멀티 태스킹**: 여러 작업이 자원을 번갈아 사용하도록 해 동시에 처리되는 것처럼 동작함

3. 다중 스레드를 지원하는 네트워크 OS

- 하나의 프로세스 안에서 여러 네트워크 작업을 동시 처리함
- 네트워크 서버로 활용 가능하며 이더넷·인터넷 연결이 안정적임
- 웹 브라우저, 메일, 뉴스, 웹 서버 등 주요 인터넷 서비스 구현에 적합함

4. 다양한 파일 시스템 지원

- 리눅스 고유 파일 시스템(**ext2-ext3-ext4**)뿐 아니라 DOS의 FAT16, 윈도우의 FAT32-NTFS, 네트워크 파일 시스템인 SMB-CIFS-NFS 등을 폭넓게 지원함

5. POSIX 표준 준수

- POSIX**는 유닉스 계열 OS가 지켜야 하는 국제 표준을 의미함
- 리눅스 역시 POSIX를 준수하므로 유닉스 계열과의 호환성이 뛰어남

6. 안정성과 효율성

- 서버·대형 시스템에서 수개월에서 수년간 재부팅 없이 동작할 만큼 안정적임
- CPU·메모리·디스크 같은 하드웨어 자원을 낭비 없이 배분해 효율적으로 구동됨

1.3 리눅스의 장점과 단점

장점	단점
- 유닉스와의 호환성이 매우 우수함 - 일반 PC용 OS보다 안정성이 높음 - 하드웨어 기능을 효율적으로 활용함 - 오픈 소스 기반으로 무료 사용이 가능함	- 공개 OS이므로 문제 발생 시 전문적인 기술 지원을 받기 어려움 - 한글 지원이 상대적으로 미흡함 - 보안 취약점이 쉽게 노출될 수 있음

1.4 리눅스의 역사

시기	사건
1965년	MIT와 벨 연구소가 MULTICS 프로젝트를 시작함. 여러 프로그램을 동시 실행하는 OS를 목표로 했지만 과도한 복잡성으로 실패함
1969년	벨 연구소의 켄 톰슨 이 MULTICS의 복잡성을 버린 단순·효율 지향 OS인 UNIX 초기 버전을 개발함

시기	사건
1971년	데니스 리치 가 C언어 를 개발하면서 어셈블리로 작성되던 UNIX가 C언어로 재작성됨. 이식성·확장성이 크게 향상됨
1985년	리처드 스톨만이 FSF(자유 소프트웨어 재단) 을 설립하고 GNU 선언문을 발표함. 오픈 소스 철학이 본격적으로 확산됨
1987년	앤드루 스투어트 타넨바움의 교육용 OS MINIX 를 개발함. 리눅스의 직접적 영감이 됨
1991년	핀란드 헬싱키 대학의 리누스 토르발스가 MINIX를 참고해 리눅스 커널 을 개발하고 인터넷에 공개함
1994년	리눅스 커널 1.0 발표됨
1996년	리눅스 커널 2.0 발표로 안정된 시스템으로 자리 잡음
현대	서버·스마트폰·슈퍼컴퓨터·차량 등 전 세계에서 가장 널리 쓰이는 OS로 성장함

1.5 주요 인물 정리

인물	기여
켄 톰슨	UNIX 공동 개발자(벨 연구소)
데니스 리치	C언어 개발, UNIX 재작성
더그 맥일로이	셸·파이프 개념 고안
리처드 스톨만	GNU 프로젝트, FSF 설립, GPL 제정, Emacs 개발
앤드루 타넨바움	교육용 OS MINIX 개발
리누스 토르발스	리눅스 커널 창시, GPL로 배포
빌 조이	vi 편집기, csh 개발
브람 무레나르	vim 편집기 개발

1.6 리눅스 배포판

리눅스는 오픈 소스이므로 누구나 커널을 가져와 기능을 덧붙여 자체 배포판을 만들 수 있음. 전 세계적으로 사용되는 주요 계열은 다음과 같음.

계열	대표 배포판	비고
레드햇(Red Hat) 계열	RHEL, CentOS, Rocky Linux, Fedora	기업·서버용으로 널리 쓰이며, CentOS 종료 후 Rocky Linux(Gregory Kurtzer)가 그 역할을 이어감
데비안(Debian) 계열	Debian, Ubuntu, Linux Mint	안정성과 커뮤니티 중심 생태계로 유명함
수세(SUSE) 계열	openSUSE, SLES	유럽 기업 시장에서 강세
슬랙웨어(Slackware) 계열	Slackware	가장 오래된 배포판 중 하나
국내 배포판	한컴리눅스, 안녕리눅스	한글 환경에 최적화됨

💡 **페도라(Fedora)**는 레드햇 계열로 분류되며, 최신 기술 검증용 역할을 함.

1.7 오픈 소스 라이선스

리눅스가 급격히 성장한 배경에는 오픈 소스 라이선스가 있음. 라이선스마다 복사·수정·배포·소스 공개 의무에 차이가 있음.

라이선스	복사	수정	배포	수정본 소스 공개 의무	특징
GPL	가능	가능	가능	필수(강한 카피레프트)	파생물 전체를 GPL로 배포해야 함. 리눅스 커널이 대표 사례
LGPL	가능	가능	가능	라이브러리 수정 시 공개	GPL보다 완화된 버전으로 독점 SW와의 링크 허용
MPL	가능	가능	가능	수정한 파일 단위로만 공개	파일 단위 카피레프트 적용
BSD	가능	가능	가능	없음	상용 제품에 포함하기 쉬움, 저작권 표시만 유지
Apache	가능	가능	가능	없음	BSD와 유사하되 특허권 조항 포함
MIT	가능	가능	가능	없음	제약이 가장 적음, 저작권 문구 유지만 요구



2. 리눅스 설치

2.1 실습 환경 준비

가상화 소프트웨어인 **VirtualBox**로 가상 컴퓨터를 생성한 뒤 그 안에 리눅스 (예: **Rocky Linux 8.8**)를 설치해 실습 환경을 구축함. 가상 머신은 호스트 컴퓨터의 CPU-메모리-디스크를 일부 할당받아 동작하므로, 호스트 사양이 충분해야 원활히 구동됨.

권장 최소 사양(가상 머신 기준)

- **CPU**: 2코어 이상(쾌적한 환경은 4코어 권장)
- **RAM**: 4GB 이상(권장 8~16GB)
- **HDD**: 20GB 이상

2.2 파티션(Partition)

하드디스크는 물리적으로 모터로 회전하기 때문에, 데이터를 안쪽·바깥쪽으로 구획 지어 저장하면 읽기·쓰기 속도를 개선할 수 있음. 이러한 구획이 파티션임.

종류	설명
주 파티션(Primary)	실제 데이터를 저장하는 기본 파티션. MBR 기준 최대 4개 까지 생성 가능
확장 파티션 (Extended)	주 파티션이 4개를 넘어야 할 때 사용하는 컨테이너 파티션
논리 파티션(Logical)	확장 파티션 안에 생성되는 세부 파티션. 개수 제한이 적음
스왑 파티션(Swap)	메모리가 부족할 때 보조 공간으로 쓰이는 전용 영역

파티션 정보는 하드디스크 맨 앞 **MBR(Master Boot Record)**에 기록되며, 저장 공간이 제한적이어서 주 파티션이 최대 4개까지만 가능함. 5개 이상의 파티션을 만들려면 **주 파티션 3개 + 확장 파티션 1개** (내부에 여러 논리 파티션) 형태로 구성함.

2.3 주요 마운트 포인트

디렉터리	용도
/boot	부트 로더와 커널 이미지 저장
/	루트 파일 시스템, 전체 디렉터리 구조의 최상위
swap	스왑 영역(전용 파티션 혹은 파일)
/home	일반 사용자 개인 디렉터리
/usr	사용자용 애플리케이션과 공유 자원
/var	로그·메일·스폴 등 가변 데이터
/tmp	임시 파일 저장 공간

2.4 디스크 장치 이름

리눅스에서 디스크·파티션의 이름은 디스크 종류에 따라 달라짐.

디스크 종류	이름 규칙	대표 용도
IDE	<code>hda</code> , <code>hdb</code> , ...	구형 PC에서 사용
SATA	<code>sda</code> , <code>sdb</code> , ...	현대 일반 PC의 표준
SCSI	<code>sda</code> , <code>sdb</code> , ...	서버용으로 널리 사용

파티션은 장치 이름 뒤에 숫자를 붙여 구분함(예: `/dev/sda1`, `/dev/sda2`). 리눅스에서는 `hda` 같은 IDE 표기법도 있었지만, 최근엔 SATA와 SCSI 모두 `sda` 네이밍으로 통일되어 표시됨.

2.5 파일 시스템(File System)

파일 시스템은 하드디스크에 파일을 어디에 어떤 데이터로 저장했는지 기록하는 체계임. 포맷(format)이란 기존 파일 시스템을 지우고 새로 만드는 작업이며, 기록이 사라져 파일이 전부 삭제된 것처럼 보임.

분류	파일 시스템	저널링	비고
리눅스	ext2	X	초기 리눅스용 파일 시스템
리눅스	ext3	O	ext2에 저널링 기능 추가
리눅스	ext4	O	ext3의 개선판, 현재 널리 사용됨
리눅스	XFS	O	고성능·고확장성 저널링 파일 시스템, CentOS 7 이후 기본
윈도우	FAT/FAT16	X	호환성이 좋은 구형 파일 시스템
윈도우	FAT32	X	FAT의 확장형, USB 등에서 사용
윈도우	NTFS	O	현대 윈도우 표준
네트워크	NFS	-	원격 파일 시스템을 로컬처럼 마운트(유닉스 계열)
네트워크	SMB / CIFS	-	윈도우-리눅스 간 파일·프린터 공유

분류	파일 시스템	저널링	비고
광학 매체	ISO9660	X	CD-ROM용 표준 파일 시스템

2.6 LVM (Logical Volume Manager)

LVM은 물리 디스크 여러 개를 논리적으로 묶어 하나의 볼륨처럼 사용할 수 있게 하는 기술임. 파티션 크기를 유연하게 확장·축소할 수 있는 장점이 있음.

구성 요소	의미
PV(Physical Volume)	LVM용으로 초기화된 실제 파티션·디스크. 가장 낮은 수준의 저장 단위
PE(Physical Extent)	PV를 구성하는 일정한 크기의 블록 단위. 기본 4MB
VG(Volume Group)	PV들을 하나의 논리적 그룹으로 묶은 것
LV(Logical Volume)	VG 안에서 할당되는 실제 사용 파티션(마운트 대상)
LE(Logical Extent)	LV를 구성하는 논리 블록 단위. PE와 1:1로 대응

구성 순서: PV → VG → LV

여러 디스크의 PE를 모아 하나의 LV로 만들면, 물리적으로는 2개 이상의 디스크지만 사용자에게는 하나의 볼륨으로 보임. 남은 PE를 이용해 LV 용량을 확장하는 것도 가능함.

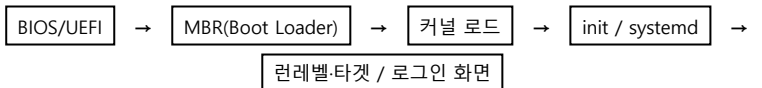
2.7 RAID (Redundant Array of Independent Disks)

여러 디스크를 묶어 성능 향상 또는 안정성 확보를 달성하는 기술임.

레벨	방식	최소 디스크	가용 용량	특징
RAID 0	스트라이핑	2개	디스크 수 × 용량	속도 최대, 안정성 없음 (하나 고장 시 전체 손실)
RAID 1	미러링	2개	디스크 수 ÷ 2	데이터 복제로 안정성 확보, 비용 부담 큼
RAID 5	스트라이핑 + 분산 패리티(1개)	3개	(디스크 수 - 1) × 용량	디스크 1개 장애 복구 가능, 쓰기 성능 다소 저하
RAID 6	스트라이핑 + 이중 패리티(2개)	4개	(디스크 수 - 2) × 용량	디스크 2개 동시 장애 허용
RAID 10	RAID 1 + RAID 0 결합	4개	디스크 수 ÷ 2	두 디스크를 먼저 미러링 후 스트라이핑, 속도·복구 모두 우수

2.8 리눅스 부팅 과정

부팅은 다음과 같은 흐름으로 진행됨.



1. **BIOS/UEFI**: 메인보드의 ROM에 저장된 펌웨어로, 전원 투입 직후 하드웨어를 점검(POST)하고 부팅 장치를 탐색함.
2. **MBR(Master Boot Record)**: 하드디스크 첫 섹터에 위치하며 부트 코드와 파티션 테이블을 포함함. BIOS는 MBR을 읽어 부트 로더를 실행함.
3. **부트 로더(Boot Loader)**: 설치된 커널 목록을 표시하고 사용자가 선택한 커널을 메모리로 로드해 실행함. 대표적으로 **GRUB** (현대 리눅스 표준)와 **LILO** (구형)가 있음.
4. **커널 초기화**: 하드웨어 장치를 인식·초기화하면서 부팅 로그를 출력함.
5. **init / systemd**: 커널이 첫 번째 사용자 프로세스를 실행함. 과거에는 `init` 이, 현대 리눅스에서는 **systemd** (PID 1)가 사용됨. systemd가 관리하는 서비스 목록은 `/etc/systemd/system`에 있음.
6. **런레벨 / 타겟**: systemd는 `runlevel3` (= `multi-user.target`, 텍스트 모드), `runlevel5` (= `graphical.target`, X 윈도우 모드) 등 부팅 타겟에 따라 서비스를 실행함. 마지막으로 로그인 화면이 나타남.

★ 부트 로더는 커널을 메모리에 적재해 실행하는 것이 주 역할이며, 다중 부팅을 지원함.

3. 기본 명령어

3.1 명령어 개념

리눅스 명령어는 사용자가 키보드로 입력해 실행하는 프로그램임. GUI의 더블 클릭도 결국 명령어를 호출하는 행위와 동일함. 입력된 명령어는 리눅스가 해당 프로그램을 실행해 결과를 화면에 출력하는 구조로 동작함. 즉 **명령어 == 실제 실행 파일(프로그램)**이며, 하드디스크에 설치되어 있어야 호출 가능함.



3.2 명령어를 위한 보조 명령어

명령어	기능
which	명령어 실행 파일의 절대 경로를 출력함(예: <code>which ls</code> → <code>/usr/bin/l</code>)
whereis	명령어의 실행 파일·소스·매뉴얼 위치까지 함께 출력함
whatis	명령어에 대한 한 줄 요약 표시함
apropos	키워드로 관련 명령어를 검색함
alias	명령어에 별칭을 부여함(예: <code>alias ll='ls -al'</code>)
unalias	설정된 별칭을 해제함
echo	문자열 변수 값을 출력함(예: <code>echo \$PS1</code>)
type	명령어가 셸 내장인지 외부 프로그램인지 구분해 알려 줌
env	현재 환경 변수 전체를 출력함

환경 변수 예시

- `PS1`: 기본 프롬프트 문자열을 지정함
- `PS2`: 보조(명령행 연장) 프롬프트
- `PATH`: 실행 파일을 탐색할 디렉터리 목록. 여기에 등록된 경로의 프로그램은 이름만으로 실행 가능함. 등록되지 않은 경로의 프로그램은 절대/상대 경로를 직접 지정해야 함
- `SHELL`, `LANG`, `HOME`, `USER`, `HISTSIZE` 등

3.3 도움말 관련 명령어

명령어 종류가 많고 옵션도 다양해 전부 외우기 어려우므로 도움말 활용이 중요함.

명령어	기능
man [명령어]	공식 매뉴얼 페이지 표시. 방향키로 스크롤, q로 종료
info [명령어]	매뉴얼보다 구조화된 문서. 하이퍼링크 방식으로 탐색 가능
명령어 --help	명령어 자체가 제공하는 간단한 사용법 출력

3.4 사용자 관리

사용자 생성·삭제·수정 명령어

명령어	설명
useradd / adduser	새 사용자 계정을 생성함
usermod	기존 계정 정보를 수정함
userdel	사용자 계정을 삭제함
passwd	사용자 패스워드를 설정·변경함
chage	패스워드 만료·최대 사용 기간 등 정책을 설정함
su - [사용자]	해당 사용자로 전환(로그인)

useradd / usermod 주요 옵션

옵션	의미
-u	UID 직접 지정
-g	기본(primary) 그룹 지정
-G	추가(secondary) 그룹 지정
-d	홈 디렉터리 경로 지정
-s	로그인 셸 지정
-m	홈 디렉터리 생성(useradd)
-c	사용자 설명(comment) 등록
-r	시스템 계정 생성(useradd)
-L	계정 잠금(usermod)
-U	계정 잠금 해제(usermod)

프롬프트로 사용자 구분

- `#`: 관리자(root)
- `$`: 일반 사용자

3.5 사용자 관련 파일

/etc/passwd — 7개 필드 구조

username : password : UID : GID : comment : home : shell

필드	내용
1	사용자 이름(username)
2	패스워드(x 표시, 실제 해시는 /etc/shadow에 있음)
3	UID(사용자 ID)
4	GID(기본 그룹 ID)
5	사용자 설명(comment, GECOS)
6	홈 디렉터리 경로
7	로그인 셸

/etc/passwd - 실제 라인 예시 (콜론 7필드)

```
root:x:0:0:root:/root:/bin/bash
ihduser:x:1000:1000:Lee Linux:/home/ihduser:/bin/bash
sshd:x:74:74:Privilege-separated SSH:/var/empty/sshd:/sbin/nologin
```

💡 식별 단서: 필드 구분자는 `:`. 2번째 필드 `x`는 "실제 해시는 /etc/shadow에"라는 표시. 7번째 필드의 `/sbin/nologin`은 로그인을 막는 시스템 계정의 시그니처(예: `sshd`, `ftp`, `nobody`).

/etc/shadow

- 암호화된 패스워드 해시를 저장함
- **root만 읽기 가능**
- 동일한 비밀번호라도 salt로 인해 서로 다른 해시가 저장됨

기타 사용자 관련 파일

파일	내용
/etc/default/useradd	useradd 실행 시 적용되는 기본값
/etc/login.defs	로그인 관련 기본 정책(비밀번호 만료·UID 범위 등)
/etc/skel	새 사용자 홈 디렉터리 생성 시 복사되는 기본 파일(예: <code>.bashrc</code> , <code>.bash_profile</code>)

3.6 그룹 관리

리눅스 사용자는 여러 그룹에 속할 수 있으며, 그룹은 파일·디렉터리 권한 부여 시 기준이 됨.

명령어	기능
groupadd	새 그룹 생성
groupmod	그룹 정보 수정
groupdel	그룹 삭제
groups [사용자]	사용자가 속한 그룹 목록 표시
id [사용자]	UID·GID·소속 그룹 등 상세 정보 표시
newgrp [그룹]	현재 세션의 기본 그룹을 변경

/etc/group 파일은 그룹 이름, 그룹 비밀번호, GID, 소속 사용자 목록을 저장함.

3.7 로그인 사용자 확인

명령어	기능
who	현재 접속 중인 사용자 목록과 접속 시간 표시
w	who 보다 상세한 정보(작업 내용, 부하 등) 표시
users	로그인 중인 사용자 이름만 간단히 표시
whoami	현재 실행 중인 사용자 이름 표시
last	최근 로그인 기록을 역순으로 표시
logname	최초 로그인한 사용자 이름 표시

3.8 리눅스 디렉터리 구조

디렉터리	용도
/	루트 디렉터리, 모든 경로의 최상위
/bin	부팅·기본 동작에 필요한 필수 실행 파일
/sbin	시스템 관리용 실행 파일(관리자 전용)



디렉터리	용도
/etc	시스템·서비스 설정 파일 모음
/usr	사용자용 애플리케이션과 공유 자원(/usr/bin, /usr/sbin, /usr/lib 등)
/var	로그·메일·스폴 등 가변 데이터
/tmp	임시 파일 저장소(재부팅 시 초기화될 수 있음)
/home	일반 사용자 개인 홈 디렉터리
/root	관리자(root)의 홈 디렉터리
/dev	장치 파일(하드디스크·터미널 등)
/proc	커널·프로세스 정보를 담은 가상 파일 시스템
/boot	부트 로더와 커널 이미지
/opt	추가 설치 애플리케이션용
/mnt	임시 마운트 지점
/media	USB·CD 등 이동식 매체 자동 마운트 지점
/lib	필수 공유 라이브러리

3.9 디렉터리 관련 명령어

명령어	기능
pwd	현재 작업 디렉터리의 절대 경로 출력
cd [경로]	지정한 디렉터리로 이동
mkdir [경로]	새 디렉터리 생성. -p 옵션으로 중간 경로까지 한 번에 생성
rmdir [디렉터리]	비어 있는 디렉터리 삭제

3.10 파일 관련 명령어

명령어	기능
ls	파일·디렉터리 목록 표시
cp [원본] [대상]	파일·디렉터리 복사
mv [원본] [대상]	파일·디렉터리 이동 또는 이름 변경
rm [대상]	파일 삭제
touch [파일명]	빈 파일 생성 또는 타임스탬프 갱신
find [경로] [조건]	조건에 맞는 파일 탐색
locate [이름]	사전에 만든 DB를 이용해 빠르게 파일 검색
file [파일]	파일 종류(텍스트·바이너리 등) 식별
ln [원본] [링크]	링크 생성(-s 옵션으로 심볼릭 링크)

ls 주요 옵션

옵션	의미
-l	자세한 정보(권한·크기·시간 등)로 출력
-a	숨김 파일(. 으로 시작)까지 모두 출력
-h	파일 크기를 KB·MB 단위로 표시
-R	하위 디렉터리까지 재귀적으로 출력
-t	수정 시간 순으로 정렬
-S	크기 순으로 정렬

cp 주요 옵션: -r (디렉터리 재귀 복사), -p (속성 보존), -f (강제), -i (덮어쓰기 전 확인)

rm 주요 옵션: -r (재귀), -f (강제), -i (확인). 관행적으로 rm -rf 로 파일·디렉터리 구분 없이 삭제함.

find 주요 옵션

옵션	의미
-name [패턴]	파일 이름으로 검색
-type [f/d/l]	파일 종류로 검색(f: 일반, d: 디렉터리, l: 링크)
-size [+/-크기]	파일 크기 조건
-mtime [일수]	수정 시각 기준 검색
-exec [명령] {} \;	찾은 파일에 대해 명령어 실행

3.11 파일 내용 확인 명령어

명령어	기능
cat [파일]	전체 내용을 그대로 출력
tac [파일]	마지막 줄부터 역순으로 출력
more [파일]	한 페이지씩 끊어 출력, 엔터로 한 줄씩 이동
less [파일]	more 의 확장, 앞뒤 스크롤 가능
head [파일]	처음부터 10줄 출력(-n 으로 줄 수 지정)
tail [파일]	마지막 10줄 출력(-f 로 실시간 추적)
wc [파일]	줄·단어·바이트 수 출력(-l, -w, -c)
diff [파일1] [파일2]	두 파일의 차이점을 라인 단위로 비교
cmp [파일1] [파일2]	두 파일을 바이트 단위로 비교
sort	내용 정렬
uniq	연속된 중복 줄 제거

3.12 표준 입출력(Standard I/O)

이름	번호	의미
표준 입력(stdin)	0	키보드 등에서 프로세스로 들어가는 입력
표준 출력(stdout)	1	프로세스가 정상적으로 내보내는 결과
표준 에러(stderr)	2	프로세스가 내보내는 에러 메시지

3.13 리다이렉션(Redirection)

입력·출력 방향을 파일이나 다른 명령어로 바꾸는 기능임.

기호	의미
>	표준 출력을 파일로 덮어쓰
>>	표준 출력을 파일 끝에 이어 붙임
<	파일을 표준 입력으로 사용
2>	표준 에러를 파일로 덮어쓰
2>>	표준 에러를 파일에 이어 붙임
&>	표준 출력과 표준 에러를 함께 파일로 보냄
	앞 명령의 표준 출력을 뒤 명령의 표준 입력으로 전달(파이프)
&	명령을 백그라운드에서 실행

예시

<pre>df -h grep / df -h > disk ls not_exist 2> err.log</pre>
--

3.14 특수 문자 / 와일드카드

문자	역할
*	임의 길이의 모든 문자열과 일치(와일드카드)
?	임의의 문자 하나와 일치
[]	대괄호 안의 문자 중 하나와 일치(예: [abc], [0-9])
~	현재 사용자의 홈 디렉터리
\	특수 문자의 의미를 제거(이스케이프)
;	명령어 구분, 앞 명령의 성공 여부와 무관하게 순차 실행
&	명령을 백그라운드 실행
&&	앞 명령이 성공할 때만 뒤 명령 실행
	앞 명령이 실패할 때만 뒤 명령 실행
\$	변수 값 참조(예: \$PATH, \$USER)
' '	작은따옴표 내부의 모든 문자는 그대로 취급(변수 치환 안 됨)
" "	큰따옴표 내부에서는 변수 치환은 유지하되 다른 특수 문자는 해제 가능



Part 2. 리눅스 운영 및 관리 ①

1. 파일 시스템 관련 명령어 ★★

리눅스는 하나의 시스템을 여러 사용자가 동시에 이용하는 구조이므로, 타인의 파일을 임의로 열람·수정·삭제할 수 없도록 권한 체계가 마련되어 있음. 본 단원에서는 이러한 권한 관리와 디스크 관리의 핵심 명령어를 정리함.

1.1 ls -l 출력 필드 해석

ls -l 명령으로 파일·디렉터리 목록을 보면 한 행에 여러 필드가 나뉘어 출력됨. 각 필드의 의미는 다음과 같음.

위치	예시	의미
① 파일 타입 + 허가권	-rw-r--r--	첫 문자는 종류, 나머지 9문자는 소유자/그룹/기타의 rwx 권한
② 링크 수	1	해당 파일·디렉터리를 가리키는 하드 링크 개수
③ 소유자명	root	파일의 소유자
④ 그룹명	grp	파일이 속한 그룹
⑤ 파일 크기	45	바이트 단위 크기
⑥ 최종 수정 시간	10월 11 13:44	마지막으로 수정된 시각
⑦ 파일명	tmp.txt	파일 또는 디렉터리 이름

허가권 10문자 중 첫 번째는 파일 종류(일반 파일 -, 디렉터리 d, 심볼릭 링크 l, 블록 장치 b, 문자 장치 c, 소켓 s 등)를 나타내고, 이후 9문자는 3자리씩 끊어서 **소유자(user) → 그룹(group) → 기타 사용자(other)**의 읽기(r)-쓰기(w)-실행(x) 권한을 표현함.

1.2 소유권 변경: chown, chgrp

- chown**: 파일·디렉터리의 소유자(및 그룹)를 변경
 - 형식: chown [옵션] 사용자[:그룹] 파일명
 - chown posein:jhkim test.txt → 소유자 posein, 그룹 jhkim으로 지정
 - chown :root BLOG → 그룹 소유권만 root로 변경
 - 옵션 -R: 하위 디렉터리까지 재귀 적용
 - 옵션 -h: 심볼릭 링크 자체의 소유권을 변경(원본 변경 방지)
- chgrp**: 그룹 소유권만 변경하는 전용 명령
 - 형식: chgrp [옵션] 그룹명 파일명
 - 옵션 -R: 하위 재귀 적용
 - 옵션 -h: 심볼릭 링크 자체의 그룹을 변경

1.3 허가권 변경: chmod

chmod는 파일·디렉터리의 접근 권한을 변경하는 명령으로, **숫자 모드**와 **기호 모드** 두 가지 방식을 지원함.

숫자 모드 (8진수 모드)

각 권한에 가중치를 부여해 합산한 8진수를 사용함.

권한	기호	값
읽기	r	4
쓰기	w	2
실행	x	1

예) chmod 764 file.txt → 소유자 rwx(7), 그룹 rw-(6), 기타 r--(4)
리눅스의 기본 허가권은 파일이 **666**, 디렉터리가 **777**이며, 여기서 umask 값이 차감되어 실제 생성 권한이 결정됨.

기호 모드 (알파벳 모드)

대상	기호	연산자	권한
소유자(user)	u	+ (추가)	r
그룹(group)	g	- (제거)	w
기타(other)	o	= (지정)	x
모두(all)	a		

예) chmod g+x file.txt → 그룹에 실행 권한 추가
예) chmod o=r file.txt → 기타 사용자 권한을 읽기로 고정
옵션 -R을 사용하면 디렉터리 하위까지 재귀적으로 권한을 적용함.

1.4 umask (기본 권한 마스크)

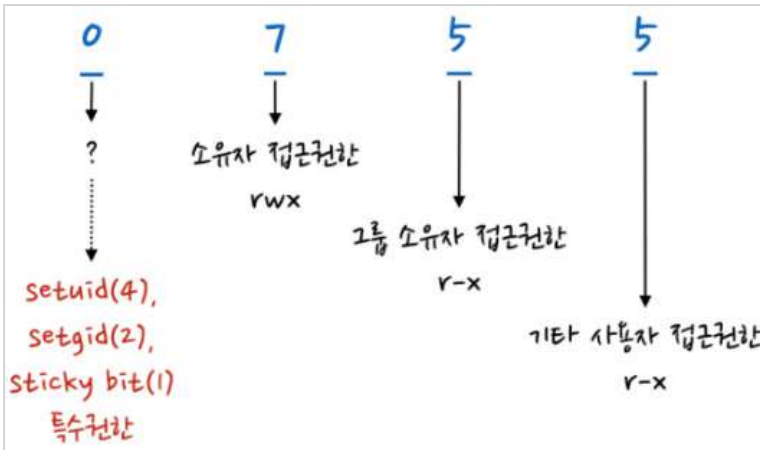
umask는 파일·디렉터리를 새로 만들 때 제외할 권한을 지정하는 값임. 기본값은 대부분 **022**이며, 실제 생성 권한은 아래와 같이 계산됨.

파일 기본 권한: $666 - umask \rightarrow 666 - 022 = 644$ (rw-r--r--)
디렉터리 기본 권한: $777 - umask \rightarrow 777 - 022 = 755$ (rwxr-xr-x)

⚠ 단순 산술 차감이 아니라 권한 비트 마스크로 계산됨. umask 명령으로 값을 바꿀 수 있지만 재부팅하면 기본값으로 되돌아감. 옵션 -S를 쓰면 값을 u=rwx, g=rwx, o=rwx 처럼 기호 형식으로 출력함.

1.5 특수 권한: SetUID / SetGID / Sticky Bit

일반 rwx 외에 특별한 실행 동작을 부여하는 세 가지 특수 권한이 존재함. 숫자 모드에서는 기존 세 자리 앞에 한 자리를 추가하여 지정함.



chmod 4자리 권한 비트 분해

특수 권한	값	표시	의미
SetUID	4xxx	소유자 실행 자리 s (실행권 있음) / S (실행권 없음)	파일 실행 시 일시적으로 소유자의 권한으로 동작
SetGID	2xxx	그룹 실행 자리 s / S	파일 실행 시 소유 그룹 권한으로 동작, 디렉터리에 설정하면 내부 생성 파일의 그룹이 해당 디렉터리의 그룹으로 상속
Sticky Bit	1xxx	기타 실행 자리 t / T	디렉터리 내 파일을 소유자·root만 삭제(이름변경)할 수 있도록 제한(내용 수정 제한 아님)

- SetUID 대표 예시**: /usr/bin/passwd
비밀번호 정보는 /etc/shadow에 저장되는데 이 파일은 root만 수정 가능함. 그러나 일반 사용자도 자신의 암호를 바꿀 수 있어야 하므로, passwd 명령 실행 파일에 SetUID를 부여해 실행되는 순간만 root 권한으로 /etc/shadow에 기록할 수 있게 함.
- SetGID 활용**: 공용 작업 디렉터리에 설정해 내부에 생성되는 모든 파일이 동일한 그룹으로 묶이도록 함.
- Sticky Bit 대표 예시**: /tmp
누구나 파일을 만들 수 있지만, 자기가 만든 파일(또는 root)만 지울 수 있도록 보호하는 공유 디렉터리에 적용.

예) chmod 3770 /project → SetGID(2) + Sticky(1) = 3 + rwxrwx--- 결합.

1.6 디스크 쿼터(Disk Quota)

파일 시스템 단위로 사용자·그룹별 디스크 사용 용량과 inode 개수를 제한하는 기능.

명령어	설명
quota	현재 사용자의 쿼터 사용 현황 출력
edquota	vi 편집기를 실행해 쿼터 값을 편집 (-p 옵션으로 타 사용자 설정 복제)
setquota	편집기 없이 명령행에서 직접 쿼터 지정, 주로 -u 옵션 사용
xfs_quota	XFS 파일 시스템 전용, -x -c 옵션 조합으로 제한 설정
quotaon / quotaoff	쿼터 기능 활성화 / 비활성화
repquota	파일 시스템 전체의 쿼터 사용량을 보고서로 요약 출력
quotacheck	파일 시스템을 스캔해 쿼터 파일 생성·점검·갱신

쿼터 설정 일반 절차: quotacheck → edquota (또는 setquota) → quotaon → repquota 로 확인.

repquota 출력 예시

```
[root@www ~]# repquota /home
*** Report for user quotas on device /dev/sda3
Block grace time: 7days; Inode grace time: 7days

      Block limits            File limits
User      used    soft   hard  grace   used    soft   hard  grace
-----
root      --      0      0      0         3      0      0
alin      --     12 102400 112640        7      0      0
joon2     --     12      0      0         7      0      0
```

💡 식별 단서: **Block limits / File limits 이중 헤더** + used·soft·hard·grace 4 컬럼 반복 구조는 repquota 만의 시그니처임. quota (개인 사용량) / edquota (편집기 진입) / setquota (출력 없음) / xfs_quota (XFS 전용 대화형) 출력과 즉시 구분됨.

/etc/fstab에서 사용하는 쿼터 마운트 옵션

옵션	대상	설명
usrquota	전체 FS	사용자 쿼터 활성화 (ext 계열·일반 표준)
grpquota	전체 FS	그룹 쿼터 활성화 (ext 계열·일반 표준)
usrjquota=aquota.user + jqfmt=vfsv1	ext3/ext4	저널링 사용자 쿼터
grpjquota=aquota.group + jqfmt=vfsv1	ext3/ext4	저널링 그룹 쿼터
uquota / quota	XFS	XFS 전용 사용자 쿼터
gquota	XFS	XFS 전용 그룹 쿼터
prjquota	XFS	XFS 프로젝트 쿼터

⚠️ 무효 옵션(혼동 주의): userquota, groupquota 는 정식 fstab 옵션이 아님. 항상 usrquota·grpquota 를 사용.

파일 시스템	저널링	주요 특징
ext2	X	리눅스 초기 표준, 파일명 최대 255자, 단일 파일 2GB, 디스크 4TB
ext3	O	ext2에 저널링 기능과 ACL을 추가
ext4	O	ext3 개선판, 대용량·고성능 지원
XFS	O	SGI가 개발한 고성능·고확장성 저널링 FS, Rocky/CentOS 기본
ReiserFS	O	작은 파일 처리에 유리한 저널링 FS
JFS	O	IBM이 개발한 저널링 FS
btrfs	O	스냅샷, 체크섬 등을 지원하는 차세대 리눅스 FS
FAT / FAT32 / vfat	X	윈도우·USB 등에서 널리 사용되는 비저널링 FS
NTFS	O	윈도우 계열의 대용량 파일 지원 FS
NFS	-	네트워크 파일 공유용 시스템(유닉스·리눅스 간)
SMB / CIFS	-	윈도우 공유 폴더 연동용, SMB를 확장한 것이 CIFS
ISO9660	-	CD-ROM 표준 FS
UDF	-	DVD/블루레이 FS, 리눅스에서 iso9660으로도 마운트 가능

1.8 디스크 추가 및 파티션 관리: fdisk

fdisk 는 파티션 테이블을 생성·삭제·수정하는 대화형 명령임. 명령행 옵션과 내부 명령이 구분됨.

명령행 옵션: -l (파티션 테이블 출력), -v (버전), -s (파티션 크기), -u (섹터 단위 표시)

키	동작
m	사용 가능한 명령 도움말 표시
p	현재 파티션 테이블 출력(print)
n	새 파티션 생성(new)
d	파티션 삭제(delete)
t	파티션의 파일 시스템 타입 변경(type)

키	동작
l	지원되는 파티션 타입 목록 조회(list)
a	부팅 가능 플래그 토글
w	변경 내용을 디스크에 기록하고 종료(write)
q	변경 사항 저장 없이 종료(quit)

파티션 속성 값: **Linux(83), Swap(82), LVM(8e), RAID(fd)**.

1.9 파일 시스템 생성: mkfs / mke2fs

- mkfs:** 파일 시스템을 생성(포맷)하는 범용 명령. -t 옵션으로 유형을 지정하거나 mkfs.xfs 처럼 확장 형태로 호출.
- mke2fs:** ext2/ext3/ext4 전용 생성 명령
 - j 옵션: 저널링 활성화(자동으로 ext3 생성)
 - t ext4 /dev/sdb1 : ext4 파일 시스템 생성
 - b : 블록 크기 지정

1.10 마운트 관리: mount / umount

생성된 파일 시스템을 실제로 사용하려면 특정 디렉터리에 연결해야 하며, 이 연결 작업을 마운트라고 함.

- mount** [옵션] 장치 마운트지점
 - t : 파일 시스템 유형 지정 (예: mount -t ext4 /dev/sdb1 /data)
 - o : 추가 옵션 지정 (ro, rw, remount, noexec, usrquota, grpquota 등)
 - a : /etc/fstab 에 정의된 항목을 모두 마운트
 - 예: mount -o remount,rw /home → 이미 마운트된 /home을 읽기·쓰기로 재마운트
- umount** : 연결 해제. USB·플로피 등 이동식 매체는 반드시 이 명령으로 해제 후 제거해야 데이터 손실을 방지함. eject 명령 수행 시 내부적으로 umount가 먼저 수행됨.

1.11 /etc/fstab 과 /etc/mtab

부팅 시 자동으로 마운트되어야 할 파일 시스템 정보는 /etc/fstab 에 기록됨. 이 파일은 **6개 필드**로 구성됨.

[장치명] [마운트 지점] [파일 시스템 타입] [옵션] [dump] [fsck 순위]

필드	설명
① 장치명	장치 경로, UUID, LABEL, 네트워크 주소, 파일명 등
② 마운트 지점	연결할 디렉터리
③ 파일 시스템 타입	ext4, xfs, vfat, nfs 등
④ 옵션	defaults, ro, rw, noexec, usrquota 등
⑤ dump	백업 주기 (0=사용 안 함, 1=매일, 2=이틀에 한 번)
⑥ fsck 순위	무결성 검사 순서 (0=검사 안 함, 1=루트, 2=일반)

fstab 을 잘못 설정하면 부팅이 실패할 수 있어 주의가 필요함. 현재 실제로 마운트된 정보는 /etc/mtab 에서 확인할 수 있음.

/etc/fstab - 부팅 시 자동 마운트 정의 (공백 구분, 6필드)

/dev/sda1	/	ext4	defaults	1	1
/dev/sda2	/home	ext4	defaults	1	2
/dev/sda3	swap	swap	defaults	0	0
UUID=7e14...	/data	xfs	defaults,noexec	0	2

/etc/mtab - 현재 마운트된 파일 시스템 (옵션을 쉼표로 나열)

/dev/sda1	/	xfs	rw,seclabel,relatime,attr2,inode64,noquota	0	0
-----------	---	-----	--	---	---

💡 **fstab vs mtab 식별 단서:** 둘 다 6필드 구조이지만 — **fstab**은 공백 정렬·관리자가 수기 작성·옵션이 defaults 처럼 단순, **mtab**은 한 줄로 나열·커널 자동 갱신·옵션이 rw,seclabel,relatime,attr2,... 처럼 길고 쉼표 구분. 기출 exam8 Q7은 mtab 형식을 보여주고 파일명을 식별하게 함.

1.12 디스크 사용량 확인: du / df

- du(disk usage):** 특정 디렉터리·파일이 차지하는 용량 측정
 - s : 전체 합계만 출력
 - h : KB/MB/GB 단위로 가독성 있게 출력
 - 예: du -sh ~ihduser → ihduser 홈 디렉터리 사용량
- df(disk free):** 파일 시스템별 남은 용량 확인
 - h : 사람이 읽기 쉬운 단위로 출력
 - T : 파일 시스템 유형까지 함께 출력
 - i : inode 사용량 출력



- o -a: 0블록 파일 시스템 포함 모든 항목 출력

df 출력 예시

```
[root@www ~]# df
Filesystem      Size  Used Avail Use% Mounted on
devtmpfs        1.9G   0    1.9G   0% /dev
tmpfs           1.9G   0    1.9G   0% /dev/shm
tmpfs           1.9G  9.4M    1.9G   1% /run
tmpfs           1.9G   0    1.9G   0% /sys/fs/cgroup
/dev/sda1       47G   8.2G   39G  18% /
tmpfs          379M  24K   379M   1% /run/user/0
/dev/sda3       1.9G  33M   1.9G   2% /mnt
```

💡 식별 단서: **Filesystem / Size / Used / Avail / Use% / Mounted on**
6컬럼 헤더는 df의 시그니처임. du (디렉터리별 용량 — 헤더 없음, 2컬럼) /
mount (on / type 형식) / fdisk (파티션 정보)와 즉시 구분됨.

1.13 블록 장치 및 UUID 확인: lsblk / blkid

- **lsblk**: 블록 장치를 트리 형태로 나열
- **blkid**: 파티션의 UUID와 파일 시스템 유형을 조회

lsblk 출력 예시

```
[posein@www ~]$ lsblk
NAME        MAJ:MIN RM  SIZE RO TYPE MOUNTPOINT
sda          8:0    0 60.6G  0 disk
└─sda1       8:1    0 46.6G  0 part /
└─sda2       8:2    0  3.7G  0 part [SWAP]
└─sda3       8:3    0    1K  0 part
└─sda5       8:5    0   1.9G  0 part
└─sda6       8:6    0   1.9G  0 part
└─sda7       8:7    0   1.9G  0 part
sr0         11:0    1 1024M  0 rom
```

💡 식별 단서: **|└┬ 트리 + MAJ:MIN RM SIZE RO TYPE MOUNTPOINT 헤더**
조합은 lsblk만의 시그니처임. blkid (UUID/TYPE 줄단위), fdisk -l (섹터·실린더), df (Filesystem/Use% 헤더)와 즉시 구분됨.

blkid 출력 예시

```
[root@www ~]# blkid
/dev/sda1: UUID="7e14520d-725e-424b-917e-ccdac407276f" TYPE="xfs"
/dev/sda2: UUID="06dc516e-68cd-401f-af90-d72576803484" TYPE="swap"
```

💡 식별 단서: **한 줄에 장치경로: UUID="..." TYPE="..." 형식**은 blkid의 시그니처임. lsblk (트리·MOUNTPOINT), fdisk -l (섹터·실린더), uuid (존재하지 않는 명령)와 즉시 구분됨.

1.14 파일 시스템 점검·복구: fsck / e2fsck / xfs_repair

- **fsck**: 손상된 파일 시스템을 점검·복구하는 범용 도구. 복구된 조각은 /lost+found 디렉터리에 저장됨. /etc/fstab의 6번째 필드와 연계됨.
- **e2fsck**: ext2/ext3/ext4 전용 점검·복구 명령. -b로 대체 슈퍼블록 위치를 지정해 손상된 기본 슈퍼블록을 우회 복구할 수 있음.
- **xfs_repair**: XFS 파일 시스템 전용 복구 도구(CentOS 7에서는 fsck.xfs가 사실상 아무 동작도 하지 않으므로 실제 복구는 xfs_repair를 사용).

2. Shell ★★

셸(Shell)은 사용자가 입력한 명령을 해석해 커널에 전달하는 **명령어 해석기**임. 그림으로 표현하면 하드웨어를 감싼 커널 위에 셸이 있고, 그 위에서 응용 프로그램이 동작하는 구조임. 사용자의 입력을 커널이 이해할 수 있도록 번역한다는 점에서 사용자와 커널을 중개하는 인터페이스 역할을 수행함.

2.1 셸의 종류

셸은 크게 **본셸 계열**과 **C셸 계열**로 분류됨. 리눅스 배포판에서는 일반적으로 **bash**가 기본 셸로 사용됨.

계열	셸	경로	개발자·특징
본셸 계열	sh (Bourne Shell)	/bin/sh	1979년 벨 연구소 스티븐 본 개발, v7 유닉스의 기본 셸
본셸 계열	ksh (Korn Shell)	/bin/ksh	1983년 AT&T 데이비드 콘, 본셸 기반에 C셸 기능 통합
본셸 계열	bash (Bourne Again Shell)	/bin/bash	1989년 GNU 프로젝트, 리눅스 표준 셸
본셸 계열	zsh (Z Shell)	/bin/zsh	1990년대, 고급 커스터마이징 제공
본셸 계열	dash	/bin/dash	데비안에서 sh의 경량 대체품, POSIX 호환(history 미지원)

계열	셸	경로	개발자·특징
C셸 계열	csh (C Shell)	/bin/csh	1978년 버클리 대학 빌 조이, C 문법 차용, history-alias·작업 제어 도입
C셸 계열	tcsh (TENEX C Shell)	/bin/tcsh	1980년대, csh 확장판으로 자동 완성 강화

2.2 셸 확인 방법

- **사용 가능한 셸 목록 조회**: cat /etc/shells
- **현재 사용 중인 셸 확인**: echo \$SHELL 또는 echo \$0
- **특정 사용자의 로그인 셸 확인**: /etc/passwd 파일의 7번째 필드

/etc/passwd 각 필드는 사용자명, 패스워드, UID, GID, 사용자 설명(GECOS), 홈 디렉터리, 로그인 셸 순으로 구성됨.

2.3 셸 변경: chsh / usermod

- **chsh(change shell)**: 일반 사용자도 자신의 로그인 셸을 변경 가능
 - o chsh -l: 사용 가능한 셸 목록 표시(/etc/shells와 동일)
 - o chsh -s /bin/csh: 기본 로그인 셸을 /bin/csh로 변경
 - o chsh -u: 사용법 도움말
 - o chsh -c: 변경된 셸에 대한 추가 설명 제공
 - o **대화식 변경**: chsh (옵션 없이) 실행 시 새 셸 경로를 프롬프트로 입력받음
 - o **다른 사용자 변경(root 전용)**: chsh [사용자명] 형식으로 지정된 사용자의 셸을 대화식으로 변경 (예: chsh kaituser)
 - o -s와 병행 가능: chsh -s /bin/zsh kaituser
- **usermod**: 관리자가 사용자 계정 정보를 변경하며 셸도 바꿀 수 있음
 - o 예) usermod -s /bin/bash ihd
 - o chsh와 달리 옵션 필수이며, 대화식 프롬프트 없이 한 번에 실행

2.4 변수: 셸 변수 vs 환경 변수

변수는 값을 임시로 저장하는 이름표임. 적용 범위에 따라 두 가지로 나뉨.

셸 변수(지역 변수)	환경 변수(전역 변수)
현재 셸에서만 유효하며 자식 프로세스에는 전달되지 않음 설정: 변수명=값	현재 셸과 자식 프로세스 모두에 적용 설정: export 변수명=값 (또는 변수 선언 후 export 변수명)

관련 명령

명령	기능
set	셸 변수와 함수 포함 모든 변수 조회·등록
env	환경 변수 출력 + 변수를 임시 변경해 명령 실행
printenv	현재 셸 환경의 환경 변수 출력
export	셸 변수를 환경 변수로 승격
unset	변수 또는 함수 해제

2.5 주요 환경 변수

변수	의미
PATH	실행 파일을 탐색할 디렉터리 목록
HOME	현재 사용자의 홈 디렉터리
SHELL	로그인 시 부여받은 셸
USER / LOGNAME	현재 로그인한 사용자 계정명
PS1	주 프롬프트 형식 (사용자명, 호스트, 경로 등)
PS2	명령이 다음 줄로 이어질 때의 보조 프롬프트
PWD	현재 작업 디렉터리
OLDPWD	직전 작업 디렉터리
LANG	기본 지원 언어 (en_US, ko_KR.UTF-8 등)
TERM	터미널 종류
HISTSIZE	히스토리 스택의 최대 명령 개수
HISTFILE	히스토리가 저장되는 파일 경로
MAIL	메일 보관 파일 경로
HOSTNAME	호스트 이름
DISPLAY	X 클라이언트가 연결할 X 서버 주소



2.6 환경 설정 파일

환경 설정 파일은 로그인 또는 셸 실행 시 자동으로 읽히는 스크립트이며, 적용 범위에 따라 전역·지역으로 구분됨.

구분	파일	설명
전역	/etc/profile	모든 사용자에게 공통으로 적용되는 환경 변수와 시작 프로그램 설정
전역	/etc/bashrc	모든 사용자에게 적용되는 alias와 함수 정의
지역	~/.bash_profile	개인 사용자의 PATH 등 환경 변수 설정
지역	~/.bashrc	개인 사용자의 alias와 함수 정의, bash 실행 시마다 적용
지역	~/.bash_logout	로그아웃 시 실행되는 명령
지역	~/.bash_history	사용자가 입력한 명령 이력 저장
지역	/etc/profile.d/	응용 프로그램이 시작 시 자동 실행할 스크립트가 위치

2.7 히스토리 기능

이전에 입력한 명령을 재사용하기 위한 기능으로, history 명령으로 기록된 목록을 번호와 함께 확인할 수 있음.

단축 표기	기능
!!	가장 최근 명령을 다시 실행
!-1	!! 과 동일
!n	히스토리에서 n번째 명령 실행
!-n	현재 기준 n번째 이전의 명령 실행
!string	문자열 string 으로 시작하는 가장 최근 명령 실행
!?string?	문자열 string 을 포함한 가장 최근 명령 실행
^old^new	직전 명령에서 old를 new로 치환해 재실행

화살표 위/아래 키로도 이전 명령들을 순차 탐색할 수 있음. 관련 환경 변수로 HISTSIZE (저장 개수), HISTFILE (저장 파일), HISTFILESIZE (파일 크기)가 있음.

2.8 프롬프트 이스케이프 코드 (PS1)

PS1 값에 사용되는 주요 이스케이프 문자.

코드	의미
\u	현재 로그인 사용자명
\h	호스트명
\w	현재 작업 디렉터리의 절대 경로
\W	현재 작업 디렉터리 경로의 마지막 이름만
\\$	일반 사용자 \$, root # 표시
\d	날짜(요일 월 일 형식)
\t	24시간제 시간
\s	사용 중인 셸 이름

예) PS1='[\u@\h \W]\\$ ' → [user@host dir]\$ 형태의 프롬프트.

2.9 alias (별명)

자주 쓰는 명령이나 옵션 조합에 짧은 별칭을 지정해 두는 기능.

- 등록: alias ll='ls -l' → 이후 ll 만 입력해도 ls -l 이 실행됨
- 확인: alias 단독 실행 시 현재 설정된 별칭 목록이 표시됨
- 해제: unalias ll
- alias가 설정된 원본 명령을 그대로 실행하려면 \ls, /bin/ls, command ls 형태로 호출하면 됨

3. 프로세스 관리 ★★

프로세스는 메모리에 적재되어 실행 중인 프로그램을 뜻함. 하드디스크에 저장된 파일이 실행되면 해당 내용이 메모리에 올라와 프로세스가 되며, 커널이 PID(Process ID) 번호로 이를 관리함. 부모-자식 관계를 나타내는 PPID, 소유자를 나타내는 UID 등의 속성이 함께 기록됨.

3.1 최초 프로세스

- 과거 CentOS 6 이하에서는 init이 PID 1을 가진 최초 프로세스였음
- CentOS 7 이상부터는 systemd가 PID 1을 담당하며, 서비스 관리의 총괄 역할을 수행

시스템 부팅 시 커널은 init(또는 systemd) 프로세스를 fork 형태로 복제해 실행하며, 이후 모든 프로세스가 이 프로세스의 자손이 됨.

3.2 프로세스 유형

프로세스는 실행 위치와 동작 방식에 따라 다음과 같이 구분됨.

- 포그라운드(foreground):** 사용자 화면 앞에서 동작, 해당 작업이 끝나야 다른 명령을 실행할 수 있음
- 백그라운드(background):** 화면 뒤에서 동작, 사용자는 다른 명령을 계속 실행 가능. 명령 뒤에 &를 붙이면 바로 백그라운드로 실행됨
- 데몬(daemon):** 백그라운드에서 상주하며 서비스 요청을 처리하는 프로세스. sshd, httpd, crond, mysqld 등이 대표적이며 부팅 시 자동 실행되도록 설정됨

데몬은 실행 방식에 따라 다시 두 가지로 나뉨.

구분	특징	예시
standalone	메모리에 항상 상주하며 요청을 즉시 처리. 빈번한 요청에 적합하지만 리소스 점유가 큼	httpd, sshd
inetd / xinetd (슈퍼 데몬)	평소엔 대기하다가 요청이 올 때만 해당 서비스를 실행. 메모리 효율적, 드물게 사용되는 서비스에 적합	telnet, rlogin, finger, pop3

3.3 포그라운드 ↔ 백그라운드 전환

조작	동작
명령어 &	명령어를 백그라운드로 실행
Ctrl + Z	포그라운드 프로세스를 일시 중지 (SIGTSTP, 20)
jobs	현재 셸에서 관리하는 작업 번호·상태 조회 (Running/Stopped/Done)
fg %작업번호	지정 작업을 포그라운드로 전환
bg %작업번호	정지된 작업을 백그라운드에서 재개
Ctrl + C	포그라운드 프로세스 강제 종료 (SIGINT, 2)
Ctrl + D	터미널 세션 종료(로그아웃 시 사용)

jobs 출력 예시

```
[posein@www ~]$ jobs
[1] - Stopped      vim a.txt
[2] + Stopped      vim b.txt
[3]   Running      find / -name '*.txt' 2> /dev/null > list.txt &
```

💡 식별 단서: **[N]** +/- 상태 명령 **4컬럼 + + (현재 작업) / - (직전 작업)** 표기는 jobs만의 시그니처임. ps (PID/TTY 헤더), fg·bg (즉시 전환·출력 없음)와 즉시 구분됨.

3.4 fork / exec

셸에서 명령을 입력하면 fork와 exec 두 시스템 호출을 통해 새 프로세스가 실행됨.

fork	exec
현재 프로세스가 자신과 동일한 복제본을 새 메모리 공간에 생성. 원본은 부모 프로세스, 복사본은 자식 프로세스가 됨	fork로 만들어진 자식 프로세스가 원래의 코드에서 벗어나 지정된 명령의 프로세스로 덮어써짐. 즉, 기존 프로세스가 새로운 프로세스로 대체됨

사용자 로그인 후 셸 실행, 시스템 초기화 시 init → systemd 교체 등이 exec를 사용하는 대표 사례임.

3.5 ps (프로세스 상태 조회)

현재 실행 중인 프로세스 목록을 조회하는 기본 명령.

- ps -e: 모든 프로세스
- ps -f: 전체(full) 형식으로 상세 출력
- ps -l: long 형식, 우선순위(NI/PRI) 값 포함
- ps -ef (System V 계열), ps aux (BSD 계열): 가장 많이 쓰는 조합

ps aux 주요 출력 필드: USER(소유자), %CPU(CPU 사용률), %MEM(메모리 사용률), VSZ(가상 메모리), RSS(실제 메모리), TTY(연결된 터미널), STAT(상태), START(시작 시간), TIME(누적 CPU 시간), COMMAND(명령어).

STAT 코드

코드	의미
R	Running, 실행 중 또는 실행 가능
S	Sleeping, 이벤트 대기 중
D	Uninterruptible Sleep, 디스크 I/O 대기
T	Stopped, 정지 상태 (SIGSTOP 등)
Z	Zombie, 종료되었으나 부모가 회수하지 않음
+	포그라운드 프로세스 그룹

코드	의미
<	높은 우선순위
N	낮은 우선순위
s	세션 리더
H	다중 스레드

3.6 기타 프로세스 조화·제어 명령

- ps**tree: 실행 중인 프로세스를 부모-자식 관계의 트리로 시각화. -p (PID 표시), -a (인자 포함), -h (현재·부모 강조)
- pgrep**: 이름으로 프로세스를 검색해 PID를 반환
- top**: CPU·메모리 사용률, 로드 평균, 프로세스별 자원 점유를 실시간으로 모니터링. 실행 중 키 입력으로 제어 가능 (k: kill, p: 특정 PID 지켜보기, m: 메모리 항목 토글, s: 갱신 주기 변경, q: 종료). %CPU·%MEM은 top 특유의 출력 값이며 ps -l에는 나타나지 않음
- free**: 사용 중/유휴 메모리 및 스왑 용량 전체를 출력
- nohup**: 터미널이 종료되어도 프로세스를 계속 실행하도록 감싸는 명령. 일반적으로 출력은 nohup.out 파일에 기록됨
 - 예) nohup sleep 1000 & → 터미널을 닫아도 sleep 프로세스가 살아 있음

top 출력 예시

```
[root@www ~]# top
18:41:40 up 4:06,  2 users,  load average: 0.00, 0.01, 0.05
Tasks: 182 total,   1 running, 181 sleeping,   0 stopped,   0 zombie
%Cpu(s):  0.0 us,   0.0 sy,   0.0 ni, 99.7 id,   0.0 wa,   0.0 hi,   0.1 si,   0.0 st
KiB Mem:  4045208 total, 3228508 free,   97644 used,  1840052 buff/cache
KiB Swap: 3985532 total, 3985532 free,      0 used. 2791328 avail Mem
```

💡 식별 단서: **5줄 고정 헤더 (load average + Tasks: + %Cpu(s) + KiB Mem + KiB Swap)**는 top만의 시그니처임. ps (헤더 없이 PID/TTY/CMD 컬럼 직진), uptime (load average 한 줄만), free (Mem/Swap 표 형식)와 즉시 구분됨.

ps

tree 출력 예시

```
[root@www ~]# pstree
systemd--+-ModemManager---2*[{ModemManager}]
          |
          |--NetworkManager--2*[dhcpcd]
          |   `--2*[{NetworkManager}]
          |
          |--2*[abrt-watch-log]
          |--abrt-d
          |--accounts-daemon---2*[{accounts-daemon}]
          |--auditd--+-audispd--+-sedispatch
          |             |         |
          |             |         |--{audispd}
          |             |         |--{auditd}
          |             `--avahi-daemon---avahi-daemon
```

💡 식별 단서: **systemd--+로 시작 + ASCII 트리(-+- | - ` -) + 2*[{...}] 열** **티스레드 표기**는 pstree의 시그니처임. ps (평면 목록), top (헤더+동적 갱신)과 즉시 구분됨. fork로 생성된 부모-자식 관계를 트리로 시각화 (exam9 Q22 fork 정답 근거).

3.7 시그널(Signal)과 종료 명령

프로세스를 제어하기 위해 커널이 전달하는 통지 신호. kill -l로 전체 시그널 목록을 확인할 수 있음.

주요 시그널

번호	이름	의미	트리거·예시
1	SIGHUP	터미널 연결 끊김, 데몬 재시작	터미널 종료, 설정 재로딩
2	SIGINT	인터럽트, 현재 포그라운드 프로세스 종료	Ctrl + C
3	SIGQUIT	종료 및 코어 덤프 생성	Ctrl + W
9	SIGKILL	강제 종료 (무시·차단 불가)	kill -9 PID
10	SIGUSR1	사용자 정의 시그널 1	애플리케이션이 자체 규약으로 사용
11	SIGSEGV	잘못된 메모리 접근, 코어 덤프	세그먼테이션 폴트
12	SIGUSR2	사용자 정의 시그널 2	애플리케이션이 자체 규약으로 사용
14	SIGALRM	타이머 만료 알림	alarm() 호출
15	SIGTERM	정상 종료 요청, kill 명령 기본값	kill PID
19	SIGSTOP	강제 일시 정지 (무시 불가)	프로세스 suspend
18	SIGCONT	정지된 프로세스를 재개	bg, fg

번호	이름	의미	트리거·예시
20	SIGTSTP	터미널에서 중지	Ctrl + Z

🌟 시그널 번호는 아키텍처·배포판에 따라 미세하게 다를 수 있으며 위 값은 일반적인 리눅스 기준임.

kill / killall / pkill

명령	대상 지정 방식	설명
kill	PID	특정 프로세스에 시그널 전달. 옵션 없으면 기본 SIGTERM(15). kill %2처럼 작업 번호 사용도 가능
killall	프로세스 이름	동일한 이름의 모든 프로세스에 한꺼번에 시그널 전달
pkill	이름·패턴	이름이나 속성(사용자, TTY 등) 조건에 맞는 프로세스에 시그널 전달

예) kill -9 2219 2229 2239 → 세 개의 PID를 즉시 강제 종료.

3.8 우선순위: nice / renice

프로세스의 CPU 점유 우선순위는 **NI(Nice Value)**로 표현됨.

- 범위: **-20 ~ 19**, 기본값 **0(또는 10)**
- 값이 작을수록 우선순위가 **높음** (즉, -20이 가장 높은 우선순위)
- 일반 사용자는 NI 값을 **증가**(우선순위 하향)만 가능, NI 감소는 root 전용

명령	대상	용도
nice	프로세스명	새 프로세스 실행 시 NI 지정 (nice -n 10 명령어 또는 nice --10 명령어)
renice	PID	이미 실행 중인 프로세스 의 NI를 절대값으로 재설정 (renice -10 1222)

renice는 반드시 옵션과 함께 사용해야 하며, -p (PID, 기본), -g (그룹), -u (사용자) 대상 지정이 가능함.

3.9 스케줄링: cron / at

반복 또는 예약 작업을 자동 실행하도록 등록하는 기능.

cron

- 데몬: crond
- 시스템 설정: /etc/crontab, 사용자별 설정: /var/spool/cron/
- 편집 명령: crontab -e (편집), crontab -l (목록), crontab -r (삭제), crontab -u 사용자 (특정 사용자 대상)

crontab의 작성 형식은 **5개의 시간 필드 + 명령어로** 구성됨.

필드	의미	범위
분	Minute	0~59
시	Hour	0~23
일	Day of Month	1~31
월	Month	1~12
요일	Day of Week	0~7 (0 또는 7=일, 1=월 ... 6=토)

특수 기호: * (모든 값), , (나열), - (범위), / (간격).

예시

```
* * * * * → 1분마다 실행
30 * * * * → 매시 30분에 실행
0 0 10 * * → 매월 10일 00:00에 실행
0 7 1 1-10/2 * /bin/vipcheck.sh
→ 1월부터 10월까지 2개월 간격으로 1일 07:00에 실행
0-59/10 * * * * → 10분마다 실행
```

at

한 번만 수행되는 일회성 예약 작업을 등록하는 명령.

- at 시간으로 시각을 지정한 뒤 실행할 명령을 입력
- atq: 예약된 작업 목록 확인
- atrm 작업번호: 예약 작업 삭제

3.10 관련 보조 명령

- halt: 시스템 종료
- suspend: 실행 중 작업 일시 중지
- tail -f: 로그 파일 실시간 모니터링 (Ctrl+C로 중단)



Part 3. 리눅스 운영 및 관리 ②

1. 에디터 ★

1.1 에디터 개요

리눅스는 환경·서비스 설정을 대부분 일반 텍스트 파일로 관리하기 때문에, 설정값을 변경하려면 해당 문서를 직접 편집할 수 있어야 함. 이때 사용되는 프로그램이 **에디터(Editor)**이며, 리눅스에는 여러 종류의 텍스트 편집기가 제공됨.

1.2 에디터 종류 및 개발자

편집기	연도	개발자	특징
vi	1976	빌 조이(Bill Joy)	초창기 BSD의 기본 편집기, 모드형 편집기
emacs	1984	리처드 스톨만(Richard Stallman)	LISP 기반 확장성, 매크로 기능, 통합 환경
pico	1989	아보일 카사르(Aboli Kasar)	메모장 유사 CUI 편집기, 자유 라이선스 아님
vim	1991	브람 무레나르(Bram Moolenaar)	vi 확장판, 색상·가시성 향상
nano	1999	크리스 알레그레타(Chris Allegretta)	pico의 GPL 대체품, GNU 프로젝트
gedit	2001	파울로 매기(Paolo Maggi)	GNOME 데스크톱용 GUI 편집기, X 윈도 우 전용
kate / kwrite	2001	크리스토프 쿨만(Christoph Cullmann)	KDE 데스크톱용 GUI 편집기

- **CUI 기반 콘솔 에디터**: vi, vim, nano, emacs, pico
- **X 윈도 우 전용 GUI 에디터**: gedit, kwrite, kate
- 개발 순서 흐름: vi → (pico → nano) → gedit
- **IDLE**: 귀도 반 로섬(Guido van Rossum)이 만든 파이썬 통합 편집 환경으로, 파이썬 학습용 에디터로 자주 언급됨.

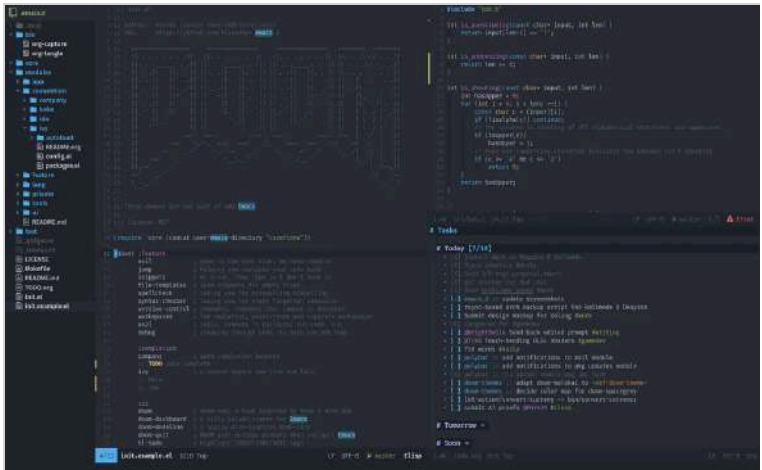
1.3 pico / nano 에디터

- **pico**: 워싱턴 대학의 아보일 카사르(Aboli Kasar)가 개발. 이메일 클라이언트(Pine)의 일부로 만들어졌으며, 자유 소프트웨어 라이선스가 아니라서 배포 제약이 있음.
- **nano**: pico의 복제판으로 GNU 프로젝트가 관리함. 라이선스가 자유롭고 사용법은 pico와 동일.
- 윈도 우의 메모장과 사용감이 비슷하여 방향키로 커서를 움직이고 바로 입력·삭제가 가능함. 각종 기능은 하단에 표시되는 단축키로 실행하며, ^ 기호는 Ctrl 키를 의미함.

단축키	기능
Ctrl + O	파일 저장 (Write Out)
Ctrl + X	편집기 종료
Ctrl + W	문자열 검색
Ctrl + G	도움말
Ctrl + K	한 줄 잘라내기 (Kill)
Ctrl + U	붙여넣기
Ctrl + A	현재 행의 맨 앞으로 이동
Ctrl + E	현재 행의 맨 끝으로 이동 (End)
Ctrl + P	커서를 윗줄로 이동 (Previous)
Ctrl + N	커서를 아랫줄로 이동 (Next)
Ctrl + Y	이전 페이지
Ctrl + V	다음 페이지
Ctrl + I	탭 삽입 (Insert Tab)

1.4 emacs

- 1976년 리처드 스톨만이 TECO 매크로 형태로 처음 선보였고, 1981년 제임스 고슬링이 C 언어로 재작성하며 Mocklisp 기반 확장 기능을 추가함. 1984년 스톨만이 Mocklisp을 실제 LISP로 대체한 GNU Emacs를 공개하면서 오늘날의 형태가 정착함.
- 비모드형(non-modal) 편집기로, 기본적으로 GUI 환경에서도 동작하며 Ctrl 또는 Alt와 다른 키를 조합해 명령을 실행함.
- 단순 편집기를 넘어 **통합 환경**을 제공하여 매크로 작성·확장 플러그인 등 확장성이 매우 높음.



GNU Emacs 실행 화면

단축키	기능
Ctrl + x → Ctrl + s	파일 저장
Ctrl + x → Ctrl + c	편집 종료
Ctrl + s	현재 내용 저장
Ctrl + f / Ctrl + b	커서를 오른쪽/왼쪽으로 이동
Ctrl + p / Ctrl + n	이전/다음 줄로 이동
Ctrl + a / Ctrl + e	줄의 처음/끝으로 이동

1.5 vi / vim 에디터

- **vi**는 1976년 빌 조이가 초기 BSD 릴리즈에 포함할 편집기로 만들었으며, **리눅스에서 가장 많이 쓰이는 기본 편집기임**.
- **vim**은 브람 무레나르가 만든 vi의 확장판으로, 색상 하이라이팅 등으로 가시성을 높임.
- 다른 편집기와 달리 **모드(Mode)** 개념이 있음.

모드	역할
명령 모드 (Command mode)	커서 이동, 복사·붙여넣기·삭제 등 편집 명령 수행. vi 실행 시 기본 진입 모드
입력(편집) 모드 (Insert mode)	실제 내용 입력·수정
ex(라스트라인) 모드 (Last-line mode)	저장, 종료, 검색, 치환, 환경설정 등 메타 명령 실행

- 어떤 모드에 있던 **Esc**를 누르면 다시 명령 모드로 복귀함.
- 명령 모드 → ex 모드로 전환할 때는 : 입력.

키	전환 방식
i	커서 위치에서 삽입 시작
I	현재 줄의 맨 처음에서 삽입
a	커서 뒤(오른쪽)에 삽입
A	현재 줄의 맨 끝에서 삽입
o	커서가 있는 줄의 아래에 새 줄 삽입
O	커서가 있는 줄의 위에 새 줄 삽입

키	기능
h / l	한 칸 왼쪽 / 오른쪽
j / k	한 줄 아래 / 위
w	다음 단어로 이동
b	이전 단어로 이동
0 또는 ^	해당 줄의 맨 앞으로 이동
\$	해당 줄의 맨 끝으로 이동

키	기능
gg	파일의 첫 줄로 이동
G	파일의 마지막 줄로 이동
nG	n번째 줄로 이동 (예: 12G)

삭제 / 복사 / 붙여넣기

- 삭제(d)**: 삭제한 내용은 자동으로 클립보드(버퍼)에 복사되어 잘라내기와 동일하게 동작함.
- 복사(y, yank)**
- 붙여넣기(p)**

명령	기능
x	커서 위치의 문자 1개 삭제
dd	현재 줄 삭제
dw	한 단어 삭제
d\$	커서부터 줄 끝까지 삭제
d5G	현재 줄부터 5번째 줄까지 삭제 (이동 키 조합)
yy	현재 줄 복사
yw	한 단어 복사
p	커서 뒤(아래)에 붙여넣기
P	커서 앞(위)에 붙여넣기
u	직전 작업 취소(undo)
r	커서 위치 문자 하나를 다른 문자로 교체 (예: ra)
c / cw	변경 / 단어 변경

d·y 는 커서 이동 명령과 조합해 "이동 범위만큼" 한 번에 삭제·복사할 수 있음. 예) dw는 한 단어 삭제, d5G는 5번째 줄까지 삭제.

검색 (명령 모드)

명령	기능
/패턴	커서 이후 방향으로 검색
?패턴	커서 이전 방향으로 검색
n	같은 방향으로 다음 결과
N	반대 방향으로 다음 결과

ex(라스트라인) 명령

명령	기능
:w	저장
:w 파일명	지정한 이름으로 저장
:q	종료
:q!	변경사항을 버리고 강제 종료
:wq, :x, ZZ	저장 후 종료
:wq!	강제로 저장 후 종료
:set nu / :set nonu	행 번호 표시 / 해제
:s/old/new/	현재 줄의 첫 번째 old를 new로 치환
:s/old/new/g	현재 줄의 모든 old를 치환
:%s/old/new/g	파일 전체 치환
:%s/^linux/Linux/	줄의 시작이 linux인 것을 Linux로 치환

치환 정규식 보조 기호

- ^ : 줄의 시작
- \$: 줄의 끝
- \< / \> : 단어의 시작 / 끝
- % : 첫 줄부터 끝 줄까지
- . : 현재 줄

vi 실행 옵션

옵션	설명
-r	손상된 파일 회복(recovery)
-R	읽기 전용 모드로 실행

옵션	설명
-c 명령	파일을 열면서 지정한 ex 명령 실행
+	파일의 맨 끝 줄에서 열기
+n 파일	n번째 줄로 커서를 두고 열기 (예: vi +10 line.txt)
+/패턴 파일	패턴이 있는 위치에 커서 (예: vi +/ihd /etc/hosts)

환경 설정 파일

- ~/.exrc : vi 편집기 환경 설정 파일
- ~/.vimrc : vim 전용 환경 설정 파일

`set` 환경 변수 (ex 모드 또는 .exrc/.vimrc)

옵션	설명
set nu / set number	행 번호 표시
set nonu	행 번호 제거
set ai / set autoindent	자동 들여쓰기
set sm / set showmatch	괄호 입력 시 대응되는 괄호 하이라이트
set ts=값 / set tabstop=값	탭 간격 지정
set ic / set ignorecase	검색 시 대소문자 무시
set list	탭·개행문자(\$) 등 특수 문자 표시
set showmode	현재 모드 표시

2. 소프트웨어 설치 ★★

2.1 프로그램 설치 방법 3가지

리눅스에서 프로그램을 설치하는 방법은 크게 세 가지가 있음.

- 저수준(패키지) 설치**: 설치 파일(.rpm, .deb)을 직접 다운로드해 전용 명령으로 설치 (예: rpm, dpkg).
- 고수준 패키지 툴 이용**: 인터넷 저장소(repository)와 연결된 상태에서 의존성까지 자동으로 해결해 주는 명령으로 설치 (예: yum, dnf, apt-get, zypper).
- 소스 코드 컴파일**: 소스코드(주로 .tar.gz) 자체를 내려받아 환경에 맞게 컴파일 후 설치. 오픈소스라는 리눅스 특성 덕에 가능한 방식으로, 소스를 직접 읽고 수정할 수 있다는 장점이 있음.

2.2 배포판별 패키지 관리 체계

배포판 계열	기본(저수준)	온라인(고수준)	확장자
레드햇 계열 (RHEL, CentOS, Rocky, Fedora)	rpm	yum / dnf (Fedora 22+, RHEL 8+)	.rpm
데비안 계열 (Debian, Ubuntu)	dpkg	apt-get, apt, aptitude	.deb
SUSE 계열 (openSUSE)	rpm	YaST, zypper	.rpm
Arch Linux	pacman	pacman	—
Gentoo	ebuild	portage, emerge	—

- alien**: 데비안에서 .rpm 파일을 .deb 파일로 변환해 주는 도구.
- YaST**: 수세의 시스템 전반 설정·관리 도구로, 패키지 관리 기능 포함(단, 순수한 패키지 관리 전담 도구는 아님).

2.3 데비안 계열

dpkg (저수준)

파일명 형식 (rpm과 달리 하이픈·언더스코어 위치가 다름)

패키지명_버전-데비안리비전_아키텍처.deb 예: cron_3.0pl1-124_i386.deb 예: nginx_1.18.0-6ubuntu14.4_amd64.deb
--

- _ (언더스코어)가 주요 구분자 (rpm은 - 하이픈 사용)
- 패키지명**: cron
- 업스트림 버전**: 3.0pl1 (원 소프트웨어 버전)
- 데비안 리비전**: 124 (패키징 버전 — 빌드 번호가 아니라 데비안 배포판의 패키징 업데이트 번호)
- 아키텍처**: i386 / amd64 / arm64 / all(아키텍처 무관)

옵션	기능
-i (--install)	패키지 설치
-r (--remove)	패키지 삭제(설정 파일은 보존)



옵션	기능
-P (--purge)	패키지 + 환경 설정 파일까지 완전 제거
-l	설치된 패키지 목록
-L [패키지]	해당 패키지가 설치한 파일 목록
-s	패키지의 버전·관리자·간략 설명 등 상태 출력
-E	이미 같은 버전이 설치되어 있으면 설치 무시

예) dpkg -r cron, dpkg -i tmp.deb

🌟 .deb 파일은 dpkg 외에도 **apt, apt-get, aptitude**로 설치/관리 가능 (rpm 계열과 유사한 계층 구조).

apt-get / apt (고수준)

명령	기능
apt-get install 패키지	의존성까지 포함해 패키지 설치
apt-get remove 패키지	패키지 삭제
apt-get purge 패키지	환경 설정 파일까지 포함 삭제 (예: apt-get purge vsftpd)
apt-get update	저장소 인덱스 갱신
apt-get upgrade	설치된 패키지 업그레이드
apt-get dist-upgrade	배포판 단위 업그레이드
apt-get autoremove	더 이상 쓰이지 않는 의존성 자동 제거

- /etc/apt/sources.list : 저장소 정보를 기록한 설정 파일로, 의존성/충돌 문제 해결 시 참조됨.
- aptitude** : APT 계열 패키지 관리를 자동화해 주는 유틸리티(우분투 등에서 사용).

2.4 레드햇 계열

rpm (저수준)

- 윈도우의 MSI/EXE에 해당하는 설치 파일로, .rpm 확장자의 파일을 rpm 명령으로 설치함.
- 의존성을 자동 해결하지 못하므로 관련 패키지가 없으면 설치 실패할 수 있음.

파일명 형식

패키지명-버전-릴리즈번호.아키텍처.rpm
예: sendmail-8.14.3-5.fc11.i586.rpm
예: nginx-1.20.1-1.el8.x86_64.rpm

주요 옵션

옵션	기능
-i (install)	패키지 설치
-v	진행 과정을 자세히 출력
-h	진행 상태를 해시(#) 마크로 표시
-U (Upgrade)	업그레이드, 설치된 버전이 없으면 신규 설치
-F (Freshen)	기존에 설치되어 있을 때만 업그레이드
-e (erase)	패키지 제거
-q	패키지 설치 여부 질의
-qa	시스템의 모든 패키지 출력
-qi	설치된 패키지 상세 정보 출력
-ql	패키지가 설치한 파일 목록 출력
-qf	특정 파일이 어떤 패키지에 속하는지 질의
-qip 파일.rpm	아직 설치되지 않은 .rpm 파일의 정보 출력
-qV / -v	패키지 검증(변조 여부 확인). S(크기), 5(MD5), T(수정일)
--nodeps	의존성 무시
--force	기존 설치와 무관하게 강제 설치

예) rpm -ivh figlet.rpm, rpm -e nmap --nodeps

검색 예시

rpm -qa | grep figlet

rpm -qa 는 설치된 전체 패키지 목록을 출력하므로, grep 과 파이프로 조합해 원하는 패키지만 골라봄.

rpm -qi 출력 예시

```
[root@www packages]# rpm -qi vsftpd
Name       : vsftpd
Version    : 3.0.2
Release    : 29.el7_9
Architecture : x86_64
Install Date : (not installed)
Group      : System Environment/Daemons
Size       : 341349
License    : GPLv2 with exceptions
Source RPM : vsftpd-3.0.2-29.el7_9.src.rpm
Build Date : Thu Jun 10 01:15:50 2021
Vendor     : CentOS
```

💡 식별 단서: **Name / Version / Release / Architecture / License 키: 값 정렬 메타데이터** 는 rpm -qi 의 시그니처임. -qa (패키지명 1줄씩 나열), -ql (파일 경로 목록), -qf (파일→패키지명 1줄 응답)와 즉시 구분됨.

yum (고수준)

- RPM의 의존성 문제를 해결하기 위해 만들어진 고수준 패키지 관리 도구.
- 지정된 인터넷 저장소(repository)에 RPM 파일을 모아 두고, 필요한 의존성 패키지까지 함께 내려받아 설치해 줌.
- 한국에서는 카카오·카이스트 등이 미리 저장소를 운영하고 있으며, 저장소에 RPM이 올라가 있지 않은 패키지는 직접 받아 설치해야 함.

명령	기능
yum install 패키지	설치 (의존성 자동 해결)
yum remove 패키지	삭제
yum update	업데이트
yum info 패키지	(미설치 포함) 패키지 정보 출력
yum search 문자열	문자열 포함 패키지 검색
yum list	전체 패키지 목록
yum list updates	업데이트 필요 목록
yum check-update	업데이트 가능 목록 확인
yum grouplist	그룹별 패키지 목록
yum groupinfo 그룹	그룹 패키지 정보
yum groupinstall 그룹	그룹 단위 설치
yum history	작업 이력 확인
yum repolist	저장소 목록 확인
yum clean	패키지 캐시 삭제
yum install epel-release	확장 패키지 저장소(EPEL) 설치

dnf

- Fedora 22부터 기본 채택된 차세대 패키지 관리 도구로, RHEL 8/Rocky Linux 8 이상에서도 사용됨.
- yum의 성능·의존성 처리 문제를 개선한 후속 도구.

2.5 소스 컴파일 설치

소스 코드는 사람이 읽는 텍스트이므로 실행 파일로 바꾸는 **컴파일(compile)** 과정이 필요함. C 언어로 작성된 경우 **GCC** 같은 컴파일러가 필요함.

컴파일 순서

- configure

→

make

→

make install
- ./configure : 현재 시스템 환경을 점검하고, 그 결과에 맞는 **Makefile**을 생성. 설치 경로는 --prefix 옵션으로 지정 가능 (예: configure --prefix=/usr/local/apache).
 - make : Makefile의 타깃과 의존성에 따라 소스 코드를 컴파일하여 실행 파일을 만들.
 - make install : 컴파일된 실행 파일·매뉴얼 등을 시스템의 정식 경로(bin, man 등)에 복사.

주요 개념

- Makefile** : make 명령이 읽어서 실행하는 빌드 스크립트. 파일 간 의존 관계와 컴파일 명령을 기록하여, 변경된 부분만 다시 빌드 가능.
- autoconf / automake** : 다양한 유닉스 환경에서 configure 스크립트와 Makefile을 자동 생성해 주는 GNU 빌드 도구.



- **CMake(Cross Platform Make)** : make 의 대체로 쓰이는 크로스플랫폼 빌드 시스템. Makefile이나 Visual Studio 프로젝트 파일까지 자동 생성. MySQL, KDE, LMMS가 CMake로 설치됨.
- **make clean** : 소스 디렉터리에서 이전에 작업한 빌드 산출물·설정 파일을 삭제하고 처음부터 다시 빌드 작업을 진행할 때 사용.

★ 참고: PHP, Apache HTTP, Nmap 등은 make 를 사용해 소스 설치.

2.6 아카이브와 압축

소스코드 배포본처럼 파일 수가 많을 때는 **아카이브(하나로 묶기)** 후 **압축(용량 줄이기)** 순으로 배포함. 아카이브 자체는 용량을 줄이지 않음.

tar	
tar [옵션] [아카이브파일] [대상파일/디렉터리...]	
옵션	기능
c	아카이브 생성 (create)
x	아카이브 해제 (extract)
t	아카이브 내 파일 목록 출력 (list)
r	기존 tar 파일에 파일 추가 (append)
v	처리 과정 상세 출력
f	아카이브 파일명 지정
P	경로 앞의 / 를 유지해 절대 경로 그대로 저장·복원
C	지정한 디렉터리로 이동 후 작업
z	gzip 압축/해제 연동 (.tar.gz)
j	bzip2 압축/해제 연동 (.tar.bz2)
J	xz 압축/해제 연동 (.tar.xz)
Z	compress 압축/해제 연동 (.tar.Z)

일반 명령과 달리 tar 는 옵션 앞의 - 를 생략할 수 있음.

주요 사용 예시

명령	설명
tar cvf a.tar dir/	dir/ 을 a.tar 로 아카이브
tar xvf a.tar	아카이브 해제
tar tvf a.tar	내용(파일 목록) 확인
tar czvf a.tar.gz dir/	아카이브 + gzip 압축
tar xzvf a.tar.gz	gzip 해제 + 아카이브 해제 동시 수행
tar jxvf file.tar.bz2	bzip2 압축 해제
tar Jxvf file.tar.xz	xz 압축 해제
tar jtvf file.tar.bz2	bzip2로 묶인 아카이브 내용만 확인
tar cvf source.tar *.c	현재 디렉터리의 C 소스를 아카이브

압축 프로그램 및 확장자

확장자	압축 / 해제	특징
.Z	compress / uncompress	가장 오래된 방식, 낮은 압축률
.gz	gzip / gunzip (또는 gzip -d)	리눅스에서 보편적
.bz2	bzip2 / bunzip2	gzip보다 높은 압축률
.xz	xz / unxz (또는 xz -d)	가장 높은 압축률
.zip	zip / unzip	윈도우와 호환

압축률 순서: compress < gzip < bzip2 < xz

gzip 주요 옵션

- gzip -d 파일 : 압축 해제
- gzip -l 파일 : 압축 파일 정보 표시
- gzip -v 파일 : 압축 과정 표시
- zcat : 압축되어 있는 텍스트 파일 내용 확인

소스 설치 실습 흐름(요약)

1. wget URL 로 .tar.gz 소스 파일 내려받기.
2. tar xzvf source.tar.gz 로 압축·아카이브 해제.
3. 필요한 컴파일러(gcc, make 등)를 yum 으로 미리 설치.

4. ./configure → make → make install 순으로 실행.

3. 장치 설정 ★

3.1 장치 인식 방식 개요

리눅스는 **PnP(Plug and Play)** 를 기반으로 장치를 자동 인식하며, 장치 파일은 /dev 아래에 생성됨. **udev**가 커널과 연동하여 장치 파일을 동적으로 생성·관리함.

주요 주변 장치

- 프린터
- 사운드 카드
- 스캐너

3.2 프린터

프린팅 시스템 패키지

- **LPRng (Line PRinter next generation)** : BSD 계열 유닉스에서 유래한 초기 리눅스 프린팅 시스템. 설정 파일은 /etc/printcap.
- **CUPS (Common Unix Printing System)** : 현재 표준으로 쓰이는 프린팅 시스템. 1999년 마이클 스위트(Michael Sweet)이 ESP에서 개발한 오픈소스이며 2007년 애플이 인수해 현재까지 유지·관리함. 설정 디렉터리는 /etc/cups.
 - **IPP(Internet Printing Protocol)** 기반의 HTTP 프로토콜로 동작.
 - TCP **631번 포트**에서 웹 서버 형태로 구동되어, 브라우저로 localhost:631에 접속해 프린터 추가·관리 가능.
 - SMB 프로토콜도 부분적으로 지원.
 - 설정 파일 문법은 Apache의 httpd.conf와 유사.
 - 장치 드라이버는 어도비의 **PPD(PostScript Printer Description)** 형식 텍스트 파일을 사용(이미지 파일 아님).
 - 프린터 등록·설정에는 lpadmin 명령을 사용.

프린터 장치 파일

- /dev/lp0 : 병렬 포트(LPT) 연결 프린터
- /dev/usb/lp0 : USB 포트 연결 프린터

프린터 설정 도구

- 레드햇 초기 : printtool, printconf
- redhat-config-printer → system-config-printer 로 대체됨 (CentOS/Rocky 등에서 GUI 프린터 설정에 사용).

프린터 관련 명령어

프린터 명령은 유닉스의 두 계열(**BSD / System V**)에서 이어져 왔으며, 현대 리눅스는 두 계열을 모두 호환 지원함.

기능	BSD 계열	System V 계열
출력 실행	lpr (/usr/bin/lpr)	lp
대기열 확인	lpq (/usr/bin/lpq)	lpstat
작업 삭제/취소	lprm	cancel requestID
프린터 제어	lpc (라인 프린터 컨트롤)	accept, reject 등

lpr 옵션

- -# : 인쇄할 매수 지정 (System V의 lp 는 -n)
- -r : 인쇄 후 관련 파일 삭제

사용 예) lpr test → test 파일을 기본 프린터로 인쇄 → lpq 로 큐 상태 확인 → lprm 으로 취소.

3.3 사운드 카드

사운드 패키지

- **OSS (Open Sound System)** : 리눅스/유닉스에서 소리를 만들고 캡처하는 인터페이스. 표준 유닉스 시스템 콜(**POSIX**)에 기반. 초기에는 자유 소프트웨어였으나 이후 사유화됨.
- **ALSA (Advanced Linux Sound Architecture)** : OSS의 사유화 이후 전환된 리눅스 표준. 사운드 카드를 자동 구성하고 장치를 관리하기 위한 리눅스 커널 요소 및 장치 드라이버를 제공.

사운드 카드 관련 명령어

명령	설명
alsactl	ALSA 사운드 카드 설정 저장·복원·제어. 예) alsactl info
alsamixer	커서(ncurses) 라이브러리 기반의 오디오 믹서(볼륨 조절)
cdparanoia	오디오 CD에서 음악 파일 추출



3.4 스캐너

스캐너 패키지

- **SANE (Scanner Access Now Easy)** : 평판 스캐너·비디오 캠 등 이미지 하드웨어를 제어하기 위한 표준화된 API(백엔드). GPL 라이선스.
- **XSANE** : SANE 인터페이스 위에서 동작하는 X 윈도우 기반 프론트엔드로, **GTK+** 라이브러리로 개발됨. 캡처 이미지 수정도 가능. 실행 명령은 **xsane**.

스캐너 관련 명령어

명령	설명
sane-find-scanner	SCSI/USB 스캐너 장치 검색
scanimage	스캐너로부터 이미지 스캔
scanadf	자동 문서 공급 장치(ADF)로 여러 장 스캔
xcam	GUI 기반으로 평판 스캐너·카메라로부터 이미지 스캔

3.5 기타 장치 관련 명령어

명령	설명
eject	CD-ROM 등 이동식 매체 트레이 꺼내기. 수행 시 umount도 자동 처리
lsmod	커널에 로드된 모듈 확인
modprobe	리눅스 커널에 모듈을 추가/제거
lsblk	블록 장치를 트리 형태로 표시
lspci	설치된 PCI 관련 장치 목록 확인
lsusb	연결된 USB 장치 목록 확인



Part 4. 리눅스 활용

1. X 윈도우 ★★

1.1 X 윈도우 개념

X 윈도우란 리눅스에서 그래픽 출력 및 마우스·키보드 입력 처리를 담당하는 GUI 시스템임. 단일 프로그램이 아닌 여러 계층의 프로그램들이 협력하는 구조로 동작함.

가장 큰 특징은 **서버-클라이언트 구조**로 설계되어 있다는 점임. 내 컴퓨터에서 실행되는 그래픽 프로그램의 화면을 다른 컴퓨터에서 볼 수도 있고, 반대로 원격 서버의 프로그램을 로컬에서 띄울 수도 있음. 이 속성을 **네트워크 투명성(Network Transparency)**이라 부르며, 네트워크 프로토콜 기반으로 동작하기 때문에 가능함. 이때 화면·입력 장치(모니터·키보드·마우스)가 있는 로컬 컴퓨터가 **X 서버**이고, 원격에서 실행되는 응용 프로그램이 **X 클라이언트**임(일반적인 직관과 반대). X 윈도우 시스템에 할당된 TCP 포트는 **6000번 이상**이 사용됨.

1.2 X 윈도우 계층 구조

아래에서 위로 갈수록 사용자에게 더 가까운 상위 추상화 계층이 위치함.

계층	역할	대표 구현
X 서버 (디스플레이 서버)	그래픽 카드·모니터 제어, 입력 이벤트 수신 및 전달	XFree86, Xorg
X 라이브러리	X 서버와 통신을 중계하는 클라이언트 라이브러리	Xlib, XCB
X 툴킷 (Xt)	메뉴·버튼·스크롤바 등 기본 위젯 API 제공	Xt
현대 그래픽 툴킷	응용 프로그램용 고수준 위젯 라이브러리	GTK+, Qt
윈도우 매니저	창 배치·이동·크기 조정·구미기 담당	Mutter, KWin 등
데스크톱 환경	작업표시줄·아이콘·파일관리자·세션 통합 제공	GNOME, KDE 등

- **Xlib**: 오래전부터 사용된 기본 클라이언트 라이브러리임.
- **XCB**: Xlib을 대체하기 위해 등장했고, 멀티스레드와 확장성이 향상됨.
- **GTK+**: GNOME 계열이 채택한 위젯 툴킷.
- **Qt**: KDE 계열이 사용하는 C++ 기반 위젯 툴킷.
- **FLTK**: 특정 데스크톱 환경에 묶이지 않는 경량 툴킷.
- **Motif**: X 윈도우 초기부터 쓰인 고전 상용 툴킷으로, CDE(Common Desktop Environment)의 기반이 되었음.

1.3 데스크톱 환경 비교

데스크톱 환경	기반 툴킷	특징
GNOME	GTK+ (LGPL)	Rocky Linux-우분투 기본, 직관적 UI
KDE	Qt (C++)	커스터마이징 자유로움, 화려한 그래픽 효과
XFCE	GTK+	경량, 생산성 지향, 낮은 리소스 사용
LXDE	GTK+	초경량, 저사양 PC 대응

1.4 윈도우 매니저

창의 배치와 표현, 열기/닫기/최소화·최대화, 이동·크기 조정을 책임지는 프로그램임. 도크, 태스크바, 프로그램 런처, 배경화면 같은 유틸리티도 제공함.

- **Mutter**: GNOME 3에서 사용하는 기본 윈도우 매니저
- **KWin**: KDE의 기본 윈도우 매니저
- **Metacity**: GNOME 2에서 사용했던 윈도우 매니저
- **Enlightenment, FVWM, TWM, Window Maker, AfterStep, Xfwm**: 그 외의 대표 매니저들

1.5 디스플레이 매니저

로그인 화면을 띄우고 사용자 세션을 관리하는 프로그램으로, 로그인 후 X 서버와 데스크톱 환경을 자동 실행시켜 줌.

- **GDM**: GNOME Display Manager
- **KDM**: KDE Display Manager
- **XDM**: X Display Manager (기본)
- **LightDM**: 경량형 디스플레이 매니저

1.6 DISPLAY 환경 변수

X 클라이언트가 어느 디스플레이로 화면을 출력할지 지정할 때 사용하는 환경 변수임. 형식은 **호스트명:디스플레이번호.스크린번호** 임.

```
echo $DISPLAY
# 예) :0.0 → 로컬 첫 번째 디스플레이, 첫 번째 스크린
# 예) localhost:10.0 → SSH X11 포워딩 환경
```

여러 GUI 세션을 동시에 띄우면 **:0**, **:1**, **:2** 와 같이 번호가 증가함. 원격지로 X 클라이언트를 전송할 때는 다음처럼 수동 설정도 가능함.

```
export DISPLAY="IP주소:디스플레이번호.스크린번호"
```

1.7 GUI 실행 및 런레벨 전환

- **startx**, **xinit**: CLI 상태에서 X 윈도우를 수동으로 기동함. xinit에 옵션 전달 시 -- 사용.
- **가상 콘솔 전환**: **Ctrl + Alt + F1 ~ F6** 조합으로 GUI와 CLI 터미널 간 전환 가능함.
- **X 윈도우 강제 종료**: **Ctrl + Alt + Backspace**

런레벨 / systemd 타킷 설정 표

구분	runlevel 방식	systemd 방식
CLI 부팅	systemctl set-default runlevel3	systemctl set-default multi-user.target
GUI 부팅	systemctl set-default runlevel5	systemctl set-default graphical.target
확인	—	systemctl get-default

부팅 모드 설정 파일로는 **/etc/inittab** 이 있음. **systemctl get-default** 의 결과가 **graphical.target** 이면 GUI, **multi-user.target** 이면 텍스트 모드로 부팅됨.

1.8 원격 X 사용

- **SSH X11 포워딩**: **ssh -X user@host** 로 접속하면 원격 서버에서 실행된 GUI 프로그램이 로컬 화면에 표시됨.
- **Xming + PuTTY 조합**: 윈도우에서 리눅스 GUI를 띄울 때 활용함.
 1. 윈도우에 Xming X 서버 설치 후 실행.
 2. 가상머신이면 호스트 포트 22 를 게스트 포트 22 로 포트 포워딩 설정.
 3. PuTTY에서 X11 포워딩 활성화 + **display :0.0** 지정 후 SSH 접속.
 4. gedit 같은 GUI 프로그램 실행 시 화면은 윈도우 측에서 렌더됨.

1.9 접근 제어 (xhost / xauth)

구분	xhost	xauth
방식	호스트(IP·도메인) 기반	토큰(MIT-MAGIC-COOKIE) 기반
용도	접근 가능한 호스트 지정·해제	X 서버 인증 쿠키 확인·관리
대표 사용	xhost +192.168.5.13	xauth list \$DISPLAY

- **xhost +**: 호스트 기반 접근 제어를 해제해 모든 호스트의 X 접속을 허용함 (**보안상 위험**).
- **xhost -**: 접근 제어를 활성화해 명시적으로 허가한 호스트만 X 서버에 접속하도록 제한함(기본 상태).
- 사용자 인증 쿠키 파일은 **~/.xauthority** 에 저장됨.

1.10 주요 X 응용 프로그램

분류	프로그램	용도
GNOME 파일관리	Nautilus	파일 관리자
GNOME 편집기	gedit	텍스트 에디터
GNOME 메일	Evolution	메일·주소록·일정
GNOME 음악	Rhythmbox	음악 재생
GNOME 동영상	Totem	동영상 재생
GNOME 이미지뷰어	Eog (Eye Of GNOME)	이미지 뷰어
KDE 파일관리	Dolphin / Konqueror	파일 관리·웹 브라우저
KDE 편집기	Kate / KWrite	텍스트 에디터
KDE 음악	Amarok	음악 재생
그래픽	GIMP	이미지 편집
그래픽	ImageMagick	커맨드라인 이미지 변환·보정
문서	Evince	PDF 뷰어
문서	LibreOffice Writer / Calc / Impress / Draw	워드/스프레드시트/프레젠테이션/드로잉
웹	Firefox, Chrome, Opera, Safari	GUI 웹 브라우저

분류	프로그램	용도
유틸	xclock, xeyes, xterm, KSnapshot	시계·눈·터미널·스크린샷

텍스트 기반 웹 브라우저인 **Lynx**, **Links**는 X 윈도우가 없는 콘솔에서도 사용 가능함.

화면 해상도 변경은 `xrandr -s 1024x768` 처럼 `xrandr` 로 수행함.

2. 인터넷 활용 ★★★

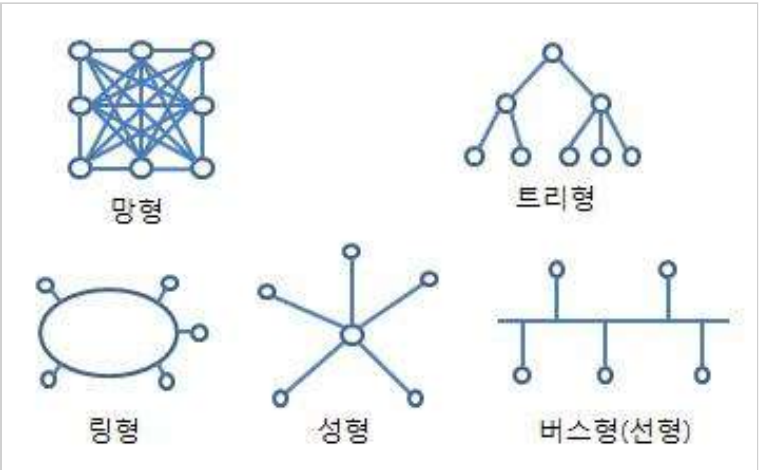
2.1 네트워크 종류

약어	풀네임	범위
LAN	Local Area Network	집·사무실·학교 등 좁은 지역
MAN	Metropolitan Area Network	도시 규모 광역, 전화·TV·인터넷 사업자망
WAN	Wide Area Network	국가·대륙·전세계
SAN	Storage Area Network	스토리지 전용 네트워크
WLAN	Wireless LAN	무선 근거리 통신망

전송 기술 측면으로는 LAN은 이더넷·Token Ring·FDDI, MAN은 DQDB, WAN은 회선교환망(PPP, HDLC)과 패킷교환망(X.25, ATM, Frame-Relay)으로 나뉨.

2.2 네트워크 토폴로지

- 버스형**: 단일 회선에 노드를 T자로 연결. 저렴하지만 노드 수 증가 시 성능 저하.
- 링형**: 노드가 원형으로 연결됨. 한 지점 장애가 전체로 파급될 수 있음.
- 스타형**: 중앙 노드(허브/스위치)에 point-to-point로 연결. 중앙 장애 시 전체 마비. **LAN 기본 토폴로지**
- 메시(망)형**: 노드들을 그물처럼 상호 연결. 장애에 강하나 설치·관리 비용이 큼. WAN에서 사용.
- 트리형**: 계층 구조로 확장된 스타 토폴로지.



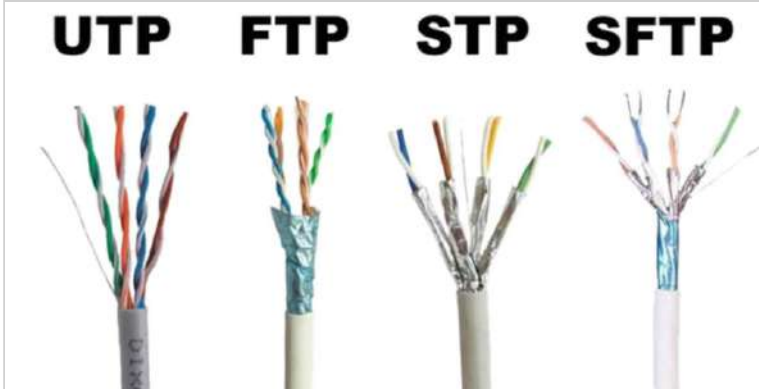
LAN 토폴로지 5종 구조

2.3 네트워크 장비와 OSI 대응

OSI 계층	대표 장비	기능
물리	허브, 리피터, 케이블(UTP·광케이블·BNC)	신호 재생·증폭, 물리적 전송
데이터링크	스위치, 브리지	프레임 전달, MAC 주소 기반 포워딩
네트워크	라우터	경로 선택, 패킷 라우팅
전송/상위	게이트웨이, 프록시	서로 다른 프로토콜 간 중계

- 리피터**: 장거리 전송 시 감쇠된 신호를 증폭·재생함.
- 브리지**: 두 LAN 세그먼트를 접속해 주는 연결 장치.
- 라우터**: 물리·데이터링크·네트워크 계층 기능을 모두 지원하여 목적지까지 가장 적절한 경로를 선택함.
- 모뎀 / DSU-CSU**: 디지털 신호와 아날로그 신호를 변환하거나 디지털 전용선 접속 시 사용.

UTP(Unshielded Twisted Pair)는 차폐되지 않은 꼬임쌍선이며, 이더넷에서 실제 사용하는 선은 8가닥 중 주파·주·녹파·녹 4가닥임. 대표 LAN 케이블 규격 요약은 다음 표와 같음.



꼬임쌍선 케이블 종류 (UTP·FTP·STP·SFTP)



BNC 동축 케이블

규격	대역폭	전송 속도
CAT-5	100MHz	100Mbps
CAT-5E	100MHz	1Gbps
CAT-6	250MHz	1Gbps
CAT-7	600MHz	10Gbps

이더넷 배선 방식에는 1000BASE-T(UTP-5), 1000BASE-LX(장파장 광섬유), 1000BASE-SX(단파장 광섬유), 100BASE-FX(광섬유 100Mbps) 등이 있음.

2.4 OSI 7계층

계층	전송 단위	주요 기능	대표 장비/프로토콜
1. 물리	Bit	전기적·기계적 특성으로 비트 전송	허브, 리피터
2. 데이터링크	Frame	동기화, 오류 제어, 흐름 제어, MAC 주소	브리지, 스위치
3. 네트워크	Packet/Datagram	라우팅, 혼잡 제어, 패킷 분할·병합	라우터 / IP, ICMP, ARP, RARP
4. 전송	Segment	end-to-end 연결·흐름·오류 제어, 신뢰성	TCP, UDP, SSL
5. 세션	Data	세션 설정·유지·종기	—
6. 표현	Data	암호화·복호화, 압축·압축해제, 표현 통일	—
7. 응용	Data	사용자와 직접 상호작용, 서비스 제공	HTTP, FTP, SMTP 등



2.5 TCP/IP 4계층 모델

실제 인터넷은 TCP/IP 4계층을 기반으로 동작하며, OSI 7계층은 이를 이론적으로 세분화한 표준임.

TCP/IP 계층	대응 OSI	역할
네트워크 인터페이스	1~2	물리 매체와 프레임 처리
인터넷	3	주소 지정·라우팅
전송	4	end-to-end 전달
응용	5~7	서비스 프로토콜

2.6 주소 체계

주소	비트 수	계층
MAC	48bit	데이터링크
IPv4	32bit	네트워크
IPv6	128bit	네트워크
Port	16bit	전송

IP 주소는 컴퓨터를 찾아가는 주소, 포트 번호는 해당 컴퓨터 안에서 특정 프로그램을 지정하는 주소임.

2.7 IP 클래스

IPv4는 1바이트(8비트) 4칸 구조로 각 칸에 0~255 사이 값이 들어감(총 32bit). 클래스 분류는 다음과 같음.

클래스	첫 비트	범위	기본 서브넷
A	0	0.0.0.0 ~ 127.255.255.255 (실사용 1.0.0.0~126.x)	255.0.0.0 (/8)
B	10	128.0.0.0 ~ 191.255.255.255	255.255.0.0 (/16)
C	110	192.0.0.0 ~ 223.255.255.255	255.255.255.0 (/24)
D	1110	224.0.0.0 ~ 239.255.255.255 (멀티캐스트)	N/A
E	1111	240.0.0.0 ~ 255.255.255.255 (예약·실험용)	N/A

2.8 사설 IP 주소

공인 IP 부족을 해소하기 위해 내부망 전용으로 쓰는 주소 대역임.

클래스	사설 IP 범위
A	10.0.0.0 ~ 10.255.255.255 (10.0.0.0/8)
B	172.16.0.0 ~ 172.31.255.255 (172.16.0.0/12)
C	192.168.0.0 ~ 192.168.255.255 (192.168.0.0/16)

2.9 특수 주소

- 127.0.0.1: 루프백(Loopback), 자기 자신 지칭. A 클래스에 속함.
- 0.0.0.0: 자신의 IP를 모를 때 또는 디폴트 경로 표현.
- 255.255.255.255: 제한적 브로드캐스트 주소.
- 각 네트워크의 첫 번째 주소는 **네트워크 주소**, 마지막 주소는 **브로드캐스트 주소**로 예약됨.

2.10 서브네팅

서브넷 마스크는 IP의 네트워크 부분과 호스트 부분을 구분함. 이진수로 앞에서부터 연속된 1이 네트워크, 뒤쪽 0이 호스트 영역을 의미하며, CIDR 표기 /n 로도 표현함.

예) C 클래스에서 서브넷 마스크가 255.255.255.192 라면 $256 - 192 = 64$ 이므로 64개씩 쪼개져 총 4개 서브넷이 생김. 마스크 255.255.255.128 인 경우 서브넷 비트 1개 / 호스트 비트 7개 → 서브넷 2개, 서브넷당 사용 가능 호스트 $2^7 - 2 = 126$ 개, 총 $2 \times 126 = 252$ 개 사용 가능함.

2.11 IPv6

- 총 128비트 길이, 16진수 8그룹을 콜론으로 구분하여 표기함.
- IPv4 주소 고갈 문제 해결 목적으로 설계됨.
- 패킷 크기 확장, 클래스 개념 제거, 패킷 출처 인증 및 비밀 보장(IPSec 내장), 흐름 제어(QoS) 기본 지원.

2.12 프로토콜 분류표

계층	대표 프로토콜
네트워크	IP, ICMP(핑), IGMP, ARP(IP→MAC), RARP(MAC→IP)
전송	TCP, UDP, SSL
응용	HTTP, HTTPS, FTP, SFTP, Telnet, SSH, SMTP, POP3, IMAP, DNS, DHCP, SNMP, NFS, NTP, LDAP, TFTP

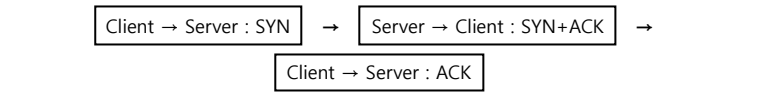
포트 번호 정리

포트	서비스
20 / 21	FTP 데이터 / 제어
22	SSH / SFTP
23	Telnet
25	SMTP
53	DNS
67 / 68	DHCP
69	TFTP
80	HTTP
110	POP3
123	NTP
143	IMAP
161 / 162	SNMP
389	LDAP
443	HTTPS
993	IMAPS
995	POP3S
2049	NFS
6000+	X 윈도우

2.13 TCP vs UDP 비교

항목	TCP	UDP
연결 방식	연결지향	비연결
신뢰성	보장(재전송)	비보장
속도	상대적으로 느림	매우 빠름
부하	상대적으로 높음	매우 낮음
헤더 크기	큼 (20바이트~)	작음 (8바이트)
연결 수립	3-way handshake	없음

2.14 TCP 3-way Handshake



2.15 TCP 상태 값

- LISTEN**: 서버가 접속 요청을 대기 중인 상태 (포트 오픈).
- SYN_SENT**: 클라이언트가 SYN을 보낸 상태.
- SYN_RECEIVED**: 서버가 SYN을 받고 응답했지만 최종 ACK 대기 상태.
- ESTABLISHED**: 3-way handshake 완료, 데이터 송수신 가능 상태.
- TIME_WAIT**: 종료 후 지연 패킷 처리를 위해 잠시 대기하는 상태.
- CLOSE_WAIT**: 상대가 연결 종료 요청 후 애플리케이션의 close를 기다리는 상태.

2.16 네트워크 서비스

WWW

웹 브라우저(클라이언트)가 웹 서버로 HTTP/HTTPS 요청(URL)을 보내면, 서버는 HTML·이미지 등 리소스를 응답으로 전달함. 대표 웹 서버는 **Apache httpd**, **Nginx**임.

메일 서비스

- MUA**(Mail User Agent): 메일 작성·열람용 클라이언트.
- MTA**(Mail Transfer Agent): 메일을 서버 간에 중계 전송.
- MDA**(Mail Delivery Agent): 수신한 메일을 사용자 메일박스에 저장.
- 송신 프로토콜: **SMTP(25)**.
- 수신 프로토콜: **POP3(110)**, **IMAP(143)** — POP3는 서버에서 메일을 내려 받고 삭제하는 편, IMAP은 서버에 남겨두고 동기화함.
- 전자우편 서비스 전체 제공에는 SMTP + POP3 + IMAP 모두가 함께 사용됨.

FTP

파일 전송용 프로토콜. 동작 모드는 두 가지임.



액티브 모드	패시브 모드
클라이언트가 서버 21 번 포트로 제어 연결을 열고, 자신이 수신할 데이터 포트 번호를 서버에 알려주면, 서버가 해당 포트로 접속하여 데이터를 전송함.	클라이언트 방화벽 등으로 역접속이 곤란한 경우 사용함. 클라이언트가 21 번 포트로 패시브 모드 요청 → 서버가 데이터용 포트를 열어 알려줌 → 클라이언트가 해당 포트로 접속하여 데이터 수신.

DNS

도메인 이름을 IP 주소로 변환해 주는 서비스임. 계층적 구조로 동작함.

- 클라이언트가 로컬 DNS 서버에 `www.youngjin.com` 질의.
- 로컬 DNS 서버가 루트 DNS 서버에 `.com`에 해당하는 TLD 서버 주소 질의.
- `.com` DNS 서버가 `youngjin.com` 권한 DNS 서버 주소 응답.
- 권한 DNS 서버가 `www.youngjin.com` 의 IP 응답.
- 로컬 DNS 서버가 클라이언트에 최종 IP 반환, 클라이언트가 해당 IP로 접속.

도메인 해석 전 `/etc/hosts` 파일을 먼저 탐색함. DNS 질의는 주로 UDP를 사용함.

Telnet vs SSH		
항목	Telnet	SSH
암호화	없음(평문)	암호화 통신
기본 포트	23	22
보안	취약, 스니핑 가능	안전
익명 접속	가능	불가
인증 키 파일	—	<code>/home/[USER]/.ssh/authorized_keys</code>

Telnet 관련 메시지 파일

- `/etc/motd`: 로그인 후 출력.
- `/etc/issue`: 로컬 로그인 전 출력.
- `/etc/issue.net`: 원격 로그인 전 출력.

SSH 주요 옵션은 `-p`(포트 지정), `-l` 또는 `user@host`(사용자 지정), `-i`(키 파일), `-L/-N`(포트 포워딩) 등이 있음.

NFS

유닉스·리눅스 계열 간에 네트워크를 통해 원격 디렉터리를 자기 컴퓨터의 파일 시스템으로 마운트해 사용하는 공유 서비스임. 포트는 **2049번**.

SAMBA

윈도우 ↔ 리눅스/유닉스 간 파일·프린터 공유 프로토콜로, 예전 **SMB** 프로토콜을 계승하여 최근에는 **CIFS**를 사용함. 리눅스·유닉스 상호 간에는 NFS, 윈도우가 끼면 SAMBA.

2.17 리눅스 네트워크 설정 파일	
파일 경로	용도
<code>/etc/hosts</code>	IP-도메인 1:1 매핑 (DNS 질의 전 우선 탐색)
<code>/etc/resolv.conf</code>	<code>nameserver</code> 지시자로 DNS 서버 지정, 검색 도메인 단축
<code>/etc/sysconfig/network</code>	호스트명, 게이트웨이, 네트워크 사용 여부
<code>/etc/sysconfig/network-scripts/ifcfg-ens33</code>	인터페이스별 IP·서브넷·게이트웨이·DNS 설정
<code>/etc/services</code>	포트 번호와 서비스 이름 매핑
<code>/etc/protocols</code>	사용 가능한 프로토콜 번호·이름 매핑

이더넷 인터페이스명은 배포판 버전에 따라 표기 방식이 다름. CentOS 6까지는 `eth0`, `eth1` 형식을 썼으나 CentOS 7부터는 버스 위치 기반 네이밍으로 바뀌어 `enp0s3`, `ens33` 처럼 장치 슬롯 번호가 포함된 이름이 쓰임.

2.18 DHCP vs 고정 IP	
DHCP	고정 IP
공유기 등 DHCP 서버에서 IP를 자동 할당 받음. 수시로 IP가 변동 가능.	서버용 컴퓨터에서는 IP가 바뀌지 않도록 수동 설정. 설정 파일의 <code>BOOTPROTO=static</code> 으로 지정하고 IP 주소, 서브넷 마스크, 게이트웨이, DNS를 모두 지정해야 함.

수동 설정 예

- IP 주소: `10.0.2.x` (VirtualBox 기본 대역)
- 서브넷 마스크: 네트워크 장치 환경에 맞춤
- 게이트웨이: 공유기(또는 가상 네트워크 게이트웨이) 주소
- DNS: 예) Google `8.8.8.8`

설정 후에는 NetworkManager 재시작으로 반영하고, `ip addr` 또는 `ifconfig` 로 IP를 확인함. 실연결 점검은 도메인으로 `ping` 을 날리면 DNS까지 함께 검증됨.

`ping google.com`

2.19 네트워크 명령어	
명령	기능
<code>ifconfig</code>	인터페이스 정보 출력, IP 설정, 활성화/비활성
<code>ip addr / ip route / ip link</code>	주소·라우팅·링크 관리 (ifconfig-route-arp 대체)
<code>route</code>	라우팅 테이블 확인 및 게이트웨이 확인
<code>netstat</code>	네트워크 연결 상태 확인 (<code>-r</code> 라우팅, <code>-a</code> 전체, <code>-n</code> 숫자, <code>-t</code> TCP)
<code>ss</code>	소켓 통계, netstat 대체 (<code>-n</code> , <code>-a</code> , <code>-t</code> , <code>-o</code>)
<code>ping</code>	ICMP 기반 호스트 연결 확인
<code>traceroute</code>	경유 경로 추적
<code>nslookup / dig / host</code>	DNS 질의
<code>arp</code>	ARP 테이블 확인 (<code>-a</code> 캐시 정보)
<code>hostname</code>	호스트명 출력/변경
<code>mii-tool</code>	이더넷 케이블 연결 상태 점검
<code>ethtool</code>	이더넷 상세 정보 및 속도·듀플렉스 설정 (mii-tool의 상위 표준 도구)
<code>system-config-network</code>	GUI 기반 네트워크 설정

게이트웨이 주소 확인은 `ip`, `route`, `netstat` 중 어느 것으로도 가능함.

2.20 국제기구 / 표준	
기관	역할
ISO	국제표준화기구, OSI 7계층 제정
IEEE	전기전자공학자협회, LAN·MAN 관련 802 시리즈 표준 제정
ITU-T	국제전기통신연합, 통신 표준 정립 (CCITT 후신)
EIA	케이블(T568B) 배열 표준화 기구
TIA	통신 장비·서비스 표준 협회
ICANN	현재 IP 주소·도메인 관리 주체
IANA	초기 IP/도메인 관리 기관 (현 ICANN 산하 기능)
FSF	자유소프트웨어재단
OSF	오픈소프트웨어재단
LF	Linux Foundation, 리눅스 표준·생태계 관리

IEEE 802 시리즈	
표준	내용
802.3	이더넷 (MAC 방식: CSMA/CD — 회선 상태를 감지하고 충돌시 재전송)
802.4	토큰 버스
802.5	토큰 링
802.6	MAN(DQDB)
802.11	무선 LAN (Wi-Fi)
802.15	블루투스·WPAN

2.21 기타 개념

- 프로토콜 구성 요소: **구문(Syntax)**, **의미(Semantics)**, **순서(Timing)**.
- 패킷 교환 방식 특징: 고정 대역 미할당, 패킷별 우선순위 부여 가능, 오버헤드 비트 존재, 이론상 무제한 호스트 수용, 회선 교환 대비 지연 증가 가능.
- IRC**: 실시간 채팅 프로토콜.
- Gopher**: 초기 텍스트 기반 정보 검색 프로토콜.
- Usernet**: 텍스트 기사 기반 분산 토론 네트워크.
- NIS(Network Information Service)**: 네트워크 사용자·호스트 정보를 중앙 집중 관리.



3. 응용분야 ★

3.1 리눅스 활용 영역

리눅스는 오픈소스와 높은 호환성 덕분에 여러 분야에서 활용됨.

- **서버**: 과거 유닉스가 주류였던 서버 분야를 리눅스가 대체하면서 웹.메일.DB.클라우드 백엔드 등을 장악함.
- **데스크톱**: GNOME·KDE 기반 일반 사용자 환경.
- **임베디드**: 스마트폰, 스마트워치, 스마트 TV, 휴대용 게임기 등 특수 목적 기기에 탑재. 운영체제 자체 설정 변경은 보통 보안상 제한됨. 임베디드 리눅스는 기존 RTOS 대비 메모리 요구량이 더 큼.
- **개발**: 컴파일러·에디터·버전관리 도구가 풍부한 개발 플랫폼.
- **모바일**: 안드로이드, Tizen 등 리눅스 기반 모바일 OS.

3.2 클러스터

여러 대의 컴퓨터를 한 대처럼 묶어 성능·가용성·분산 처리를 실현하는 기술임.

분류	풀네임	목적
HA Cluster	High Availability	지속적 서비스 보장. 주 노드 장애 시 백업 노드가 인계. 부하분산기와 주기 통신으로 상태 점검
HPC Cluster	High Performance Computing	대규모 연산용. 베퍼울프 클러스터, 슈퍼컴퓨터 제작 방식
LVS Cluster	Linux Virtual Server (부하분산)	여러 서버로 트래픽을 분산해 대규모 서비스 제공, 로드 밸런서 활용

로드 밸런싱 예: 동일 웹 서버가 설치된 여러 리눅스 서버 앞에 **로드 밸런서**를 두면 클라이언트에게는 한 대의 서버처럼 보이지만 실제로는 다수 서버가 요청을 나눠 처리함.

3.3 가상화

가상화란 실제 물리적 장치나 시스템을 소프트웨어로 모사하여 컴퓨터 자원(CPU·메모리·디스크·네트워크·CD/DVD 등)을 논리적으로 제공하는 기술임. Rocky Linux 설치에 쓰는 **ISO 파일**도 CD 이미지의 가상화 결과물임. 단, 물리 자원에 장애가 발생하면 가상화 서비스 자체도 동반 장애가 생기므로 가상화가 모든 문제를 해결해 주는 만능은 아님.

하이퍼바이저는 호스트의 CPU·메모리·디스크를 나눠 가상 머신에 부품처럼 제공하는 프로그램임. 방식은 두 가지임.

유형	방식	예시
Type 1 (Bare-metal)	호스트 OS 없이 하드웨어 위에 직접 동작	Xen, KVM, Hyper-V, VMware ESXi
Type 2 (Hosted)	기존 OS 위에서 애플리케이션처럼 동작	VirtualBox, VMware Workstation, Parallels Desktop, QEMU

- **KVM**: 리눅스 커널에 통합된 하이퍼바이저. x86 기반 CPU 전가상화, QEMU 에뮬레이터 사용, Intel VT·AMD-V 하드웨어 가상화 기술 활용, 이더넷 및 Disk I/O 반가상화 지원. 2006년 Qumranet이 개발해 2007년 리눅스 커널 2.6.20에 병합됨.
- **Xen**: 케임브리지 대학 출발의 오픈소스 하이퍼바이저(2003년 첫 공개). 전가상화와 반가상화 모두 지원하며, 호스트 OS 없이 Domain 0이라는 특수 도메인이 제어 역할을 담당함. 반가상화 시 다른 아키텍처의 게스트는 실행 불가.
- **VirtualBox**: InnoTek 개발 → 현재 Oracle 관리. 제한된 GPL 버전 상용/사유 소프트웨어로 기본 디스크 확장자는 `vdi`.

가상 디스크 파일 형식

형식	제공자
VMDK	VMware
VDI	Oracle VirtualBox
VHD	Microsoft Hyper-V

3.4 클라우드 컴퓨팅

인터넷을 통해 타인의 컴퓨팅 자원을 빌려 쓰는 방식임. 대표 업체인 **AWS**는 사용자 요청 시 하이퍼바이저로 가상 머신을 즉시 생성하여 제공함. 오픈소스 중에서는 **OpenStack**이 대표적인 IaaS 클라우드 플랫폼이며, Cloudstack과 OpenNebula도 하이퍼바이저 통합 관리 플랫폼으로 분류됨.

클라우드 서비스 모델 비교

모델	제공 범위	대표 예시
IaaS (Infrastructure)	서버·스토리지 등 IT 하드웨어 자원	AWS EC2, GCE
PaaS (Platform)	소프트웨어 개발·배포용 플랫폼	Heroku, Google App Engine
SaaS (Software)	실행형 응용 프로그램	Gmail, Office 365

모델	제공 범위	대표 예시
DaaS (Desktop)	가상화된 데스크톱 아웃소싱	—
FaaS (Function)	이벤트 기반 함수형 실행	AWS Lambda

사용자의 설정 자유도는 **IaaS > PaaS > SaaS** 순.

3.5 빅데이터

빅데이터는 대용량·다양성·빠른 생성 속도를 가진 데이터를 처리하는 분야임. 주요 특성은 **5V**로 정리됨.

특성	의미
Volume	규모
Variety	다양성
Velocity	속도
Veracity	정확성
Value	가치

관련 프로그램

- **Hadoop**: 대규모 데이터 분산 처리 프레임워크. 내부 **HDFS**(Hadoop Distributed File System)로 큰 파일을 여러 노드에 분산 저장하고 병렬 처리.
- **Cassandra**: 2008년 Apache 오픈소스로 공개된 분산 데이터베이스.
- **MongoDB**: NoSQL 문서 기반 DB.
- **R**: 통계 분석과 시각화에 특화된 프로그래밍 언어.

3.6 모바일 OS / 임베디드 플랫폼

OS	설명
Android	구글이 개발한 리눅스 기반 모바일 플랫폼
Tizen	인텔·삼성·리눅스 재단·Meego 개발자 주도 리눅스 기반 무료 모바일 OS
webOS	리눅스 커널 기반 임베디드 OS (LG 등에서 활용)
Firefox OS	모질라의 웹 기반 모바일 OS
Maemo	노키아의 스마트폰·인터넷 태블릿용 리눅스 플랫폼
Moblin	인텔·리눅스 재단이 만든 넷북·임베디드용 오픈소스 OS
QNX	유닉스 계열 실시간 서버 OS (1982)
iOS	애플의 아이폰 전용 OS (유닉스 계열)

★ 리눅스 기반 운영체제: webOS, Tizen, GENIVI. QNX는 유닉스 계열.

3.7 차량 / 임베디드 플랫폼

- **GENIVI**: 오픈소스 기반 차량 인포테인먼트(IVI) 플랫폼 표준화.
- **AGL (Automotive Grade Linux)**: 리눅스 재단 주도의 자동차용 리눅스 프로젝트.

3.8 싱글보드 컴퓨터 / 교육용 하드웨어

저가·소형 단일 기판 컴퓨터가 교육과 IoT 시제품 제작에 널리 쓰임. 대표 제품은 다음과 같음.

제품	분류	특징
아두이노(Arduino)	마이크로 컨트롤러	센서·액추에이터 제어에 특화된 오픈소스 보드
라즈베리파이(Raspberry Pi)	싱글보드 컴퓨터	저가·교육용으로 설계된 범용 리눅스 보드
마이크로비트(micro:bit)	초소형 컴퓨터	기울기 센서·나침반 등 내장, 어린이 코딩 교육
큐비보드(Cubieboard)	미니 PC	안드로이드·우분투 설치 가능한 소형 보드

3.9 플랫폼 · 자동화 도구

도구	분류	역할
OpenStack	IaaS 클라우드	오픈소스 프라이빗/퍼블릭 클라우드 구축·관리
Docker	컨테이너	애플리케이션 이미지 패키징·배포
Kubernetes	오케스트레이션	컨테이너화 애플리케이션의 배포·확장·관리 자동화
Ansible	자동화	구성 관리, 애플리케이션 배포, 작업 자동화





C O P Y R I G H T

저작권 안내

본 자료는 「**저작권법**」으로 보호되는 저작물입니다.
구매자 본인의 학습 목적으로만 이용할 수 있으며,
무단 **복제 · 배포 · 공유 · 2차적저작물 작성**을 금지합니다.

침해 시 **5년 이하의 징역 또는 5천만원 이하의 벌금**에 처해질 수 있으며,
영리 · 상습 침해는 고소 없이 처벌됩니다. (저작권법 §136①, §140)

본 자료에는 **디지털 워터마크와 구매자 식별정보**가 삽입되어 있어, 무단 유포 시 그 경로가 추적됩니다.